📖 TTSWAP Decentralized Mall Whitepaper

> **In one sentence**: TTSWAP is a **decentralized mall** built on the blockchain — never closing, with no boss and no cashier.
> Anyone can list goods, set up a shelf, be a customer, or be a supplier.
> The whole mall shares a single **central checkout**, and any two shelves can swap goods directly; suppliers earn fees just by stocking inventory, and commitment-type shelves can also mine TTS at the same time; trading fees are automatically split on-chain among six parties — "supplier / brand / storefront / referral / customer / mall" — according to on-chain rules.

> **Who this whitepaper is for**: non-technical readers (users / operators / merchants / investors) who want to understand TTSWAP.
> The whole text uses a "mall story" to string every mechanism together, retelling every on-chain concept in everyday language.

Table of Contents

1. Prologue: A Never-Closing Decentralized Mall
2. Mall Basics: Goods, Shelves, Supply Certificates, Hard Currency
3. The Central Checkout: Any Shelf Can Swap for Any Other Shelf
4. Listing Goods: Opening New Shelves and Brand Rights
5. Supplier Restocking: Why One Good Alone Is Enough
6. Customer Checkout: Two Ways to Buy
7. Merchants and Remote Payment (X402)
8. The Shelf Inventory "Seatbelt"
9. Supplier Settlement: How to Reclaim Principal and Profit
10. The Six-Party Fee Split: Who Gets a Cut of Each Trade
11. Governance: Who Can Reprice and Who Can Close a Store
12. The TTS Token: The Mall's Scarcest "Equity Certificate"
13. Upgrades, Circuit Breakers, and Permanent Immutability
14. Common Scenario Cheat Sheet
15. Contact & Legal License

16. Appendices: Mechanism Cheat Sheet, Parameter Limits, and Comparison with Mainstream DeFi

1 Prologue: A Never-Closing Decentralized Mall

Picture a mall built in the cloud:

- No boss, no cashier, but one computer that never makes mistakes (= a smart contract);
- No business hours — open 24 hours a day, 365 days a year;
- Every good has its own **shelf**, and customers can freely swap goods between any two shelves;
- Anyone can list a new good, or restock an existing shelf and earn returns;
- The fee on every trade is automatically split by the computer among six parties — "supplier / brand / storefront / referral / customer / mall" — according to fixed rules;
- Every user who holds TTS tokens is an owner of the mall: they enjoy the rights to oversee, propose, vote, and receive dividends.

TTSWAP is the on-chain version of this mall. This entire whitepaper is about how it runs.

1.1 Why Open a Mall Like This

In the traditional crypto world, "swapping good A for good B" usually means walking into a **specialty shop that only sells the two goods A and B**. In DEXs like Uniswap, this shop is called a **pair**. If no one has ever opened this shop on-chain, you have to go "A → USD → B" — two shops, two trips.

An intuitive comparison:

- **Traditional DEX is like a commercial street**: every pair is a **specialty shop selling only 2 goods**, stocking its own inventory, doing its own accounting, and setting its own prices;
- **TTSWAP is the large mall at the end of that street**: the whole place has just **one central checkout**, one shelf per good, and any two shelves swap directly.

The problems with a commercial street:

- **The more goods there are, the more shops you need, and the more scattered the inventory**. In theory, 100 goods require **4,950 shops** each selling only 2 goods; a single good has to be placed into 100 different shops;
- **High barrier to opening a shop**: a shop must stock its two goods at **equal value** (e.g. 1 ETH + 2000 USDT). A supplier who holds only one of them, or whose two goods don't match in value, **simply cannot open a shop**;
- If no one opens a shop for a new good, it is effectively **untradeable**;

- Customers have to research "which shop first, which shop next" themselves, paying slippage, Gas, and time twice in the middle;
- Once a supplier puts goods in one shop, they can only serve that shop's customers forever — **other shops on the street can't use that liquidity even if they want to**.

TTSWAP's approach is simple: **move all goods into the same mall and share a single central checkout**. So:

- 100 goods become 100 shelves, not 4,950 shops;
- A new good only needs to be put on a shelf, and it can immediately be swapped by all customers;
- When a supplier restocks a shelf, **that shelf's liquidity depth is used by the entire mall**;
- A customer entering the mall does only one thing: "**I want to swap this good for that good**," and the checkout settles it in a single step.

1.2 What This Whitepaper Covers

> The 5 main threads of this mall:
>
> 1. **How goods get listed** (who can open a new shelf / how the initial price is set)
> 2. **Who restocks shelves, and how restocking earns money** (supplier mechanism + impermanent-loss defense + withdrawal rules)
> 3. **How customers buy** (two order types + remote payment + anti-sandwich protection)
> 4. **How the mall lets everyone earn** (six-party fee split: supplier / brand / storefront / referral / customer / mall)
> 5. **How TTS tokens are distributed** (4C equity + doubling unlock + supply-as-mining + buyback-and-burn)

Once you've read these five threads, even if you've never written a single line of Solidity, you'll understand what **mall-world action** every key TTSWAP mechanism corresponds to.

2 Mall Basics: Goods, Shelves, Supply Certificates, Hard Currency

Before we get into trading and restocking, let's clarify the five basic concepts in the mall.

2.1 The Shelf (Pool): A Good's Only "Home"

Every good in the mall corresponds to **exactly one shelf**, and each shelf records four pieces of information:

Shelf fundamentals	Mall term	Corresponding on-chain field
Market value V	How much this shelf's good is worth in this mall	`investState.amount1()`
Current inventory Q	How many units are on the shelf right now	`currentState.amount1()`
Cumulative stock-in I	How many units all suppliers have restocked historically	`currentState.amount0()`
Total supply shares S	The total "supply shares" held by all suppliers combined	`investState.amount0()`
Total virtual quantity R	The total virtual quantity across all suppliers	`goodconfig.amount1()`

Intuitively:

- **V is each good's "sticker price," which doesn't change directly from stock-in or stock-out**;
- **Q is the current quantity of goods on the shelf** (rises and falls with trading);
- **I is how much of the shelf's inventory was "actually placed there by suppliers"** (always recorded, reflecting support depth);
- **S is the total supply shares held by all suppliers** (used as the pro-rata basis when withdrawing).
- **R is the total non-tradeable quantity on the shelf.**

Rule 1: A given good has only one shelf across the whole mall (enforced at the contract level: if `goods[goodid].owner != 0` , it cannot be opened again).

Rule 2: Each good is identified by a `T_GoodKey` triple — `(asset type, contract address, sub-id)` . A plain ERC20 token's sub-id is fixed at 0; in the future, tokenized stocks (ERC-1155) and prediction-market shares (ERC-6909) can use different sub-ids to distinguish multiple underlyings under the same contract.

2.2 The Supply Certificate (Proof of Investment)

Each time someone restocks a shelf, the mall issues a **supply certificate** that records:

```
Supply Certificate {
    Which shelf I supplied (currentgood)
    How many supply shares I hold on this shelf (shares)
    How much value my goods were converted to at the time (state)
    How many virtual display units I contributed (invest, virtual)
    How many units I actually delivered (invest, actual)
}
```

Key point: **the certificate is not a resellable NFT — it's a ledger entry uniquely bound to "identity + shelf."**

> `proofid = keccak256(owner, goodid)` — a **single-token certificate**, no longer bound to a "paired value token."

When the same person restocks the same shelf multiple times, it accumulates onto the **same certificate** (it does not become N certificates). In other words: **identity paired with shelf → exactly one certificate.**

2.3 The Two Tiers of Goods

v2 simplifies good tiering: **it no longer distinguishes between two separate pool-creation paths for "meta goods" and "hard currency."** Everything opens a shelf with `initGood`, and the admin later elevates privileges.

Tier	Who fills it	Privileges	Mall intuition
Hard Currency (Value Good)	Widely-recognized goods, elevated by the **CEO (Market Admin)** per community resolution	Can serve as a value anchor; `safeLine` may be relaxed by default	The mall's **"bestselling tier-1 goods"** (USDT / ETH / WBTC, etc. — goods customers are happy to use as money)
Normal Good	Any newly-listed good is this tier by default	Constrained by the **safeLine**: large trades can't instantly push	The mall's **long-tail goods**: anyone trying to endlessly pile in inventory and then siphon out the hard currency

Tier	Who fills it	Privileges	Mall intuition
		the shelf out of its healthy range	will be stopped by the seatbelt

Why do normal goods need safety constraints? If the mall accepted anything, you'd get "suppliers cramming in garbage goods to siphon off the mall's real USDT / ETH." The protocol uses `safeLine` upper/lower bounds (a seatbelt set at the contract level; see §8).

2.4 Why Hard Currency Is So Special

Hard currency is a "widely recognized, relatively value-stable" good hand-picked by the mall's executive layer (the CEO, who holds the Market Admin key). Once tagged as hard currency:

- Anyone restocking it is priced at the **current in-pool price**;
- It can serve as a pricing anchor, and its `safeLine` is usually configured more loosely;
- The six-party fee split is more stable on hard-currency shelves.

For a new good to be upgraded to hard currency, it must be approved by the **CEO (Market Admin)** through a Class-II community proposal, who then calls `modifyGoodByAdmin` to flip on the hard-currency flag — **it is not automatic; it requires proposal approval.**

2.5 The Mall's "Central Checkout": The Singleton Contract

The whole mall has just **one central checkout**, and every shelf plugs into this one register:

- The frontend integrates against the **Proxy contract address** (`TTSwap_Market_Proxy`) — logic is upgradeable, state is invariant;
- All shelves live under the same `mapping(goodid => S_GoodState)` , with `goodid` computed from `T_GoodKey` ;
- All trades, restocks, and withdrawals happen inside this contract; it also supports `multicall` to batch several actions in one transaction.

Benefits:

- Liquidity depth aggregates naturally (see §3);
- Low Gas consumption (one call and it's done; see the appendix);
- Small audit surface (concentrated logic, no cross-contract inconsistencies).

3 The Central Checkout: Any Shelf Can Swap for Any Other Shelf

3.1 The Pain of Traditional DEXs: N Specialty Shops on One Commercial Street

Take Uniswap: to swap bread for milk, someone must first open a **"bread ↔ milk specialty shop"** (pair) on-chain. This shop **only sells the two goods bread and milk** — not a third. If no one has opened it, you have to go "bread → USD shop → milk shop," through two shops, two swaps.

Number of goods	How many "2-good specialty shops" you need to open
3 goods	3 shops
10 goods	45 shops
100 goods	4,950 shops
N goods	N(N-1)/2 shops

The problem: **each shop stocks independently and has poor depth**, and no one wants to open a shop for long-tail goods at all.

3.2 TTSWAP's Approach: One Mall, One Central Checkout

TTSWAP stuffs the entire commercial street into one large mall, where everyone shares **the same central checkout**. When you want to swap good A for good B, the mall does this:

Step 1: The checkout first assigns a "purchasing power" to your good A

- It glances at the A shelf: how many units of A are on it right now (Q_A), and how much A is worth in total (V_A);
- It applies a formula to convert the batch of A you bring in into "internal mall purchasing power" (denoted ΔV).

Step 2: It then uses that purchasing power to buy B

- It glances at the B shelf: how many units of B are on it right now (Q_B), and how much B is worth in total (V_B);
- It applies a formula to compute how many units of B you can take away (denoted Δb).

The whole process completes in **a single contract call** — from start to finish you perform just one action: "swap A for B."

Two Underlying Formulas (for readers who want the math)

Step 1 and Step 2 are really just two algebraic expressions in the on-chain code.

Formula 1: quantity → value (you push Δa units of goods into the A shelf, converting to "internal mall purchasing power" ΔV)

$$\Delta V \;=\; \frac{2 \cdot V_A \cdot \Delta a}{2 \cdot Q_A \;+\; \Delta a}$$

Formula 2: value → quantity (use purchasing power ΔV to take how many units of B from the B shelf)

$$\Delta b \;=\; \frac{2 \cdot Q_B \cdot \Delta V}{2 \cdot V_B \;+\; \Delta V}$$

Together, the two formulas satisfy three things:

- **Value conservation**: the total value ΔV of the A you push in strictly equals the total value of the B you take out;
- **Bounded slippage**: the more you send, the worse the unit price — no "whale sweeps the shelf clean in one shot";
- **O(1) constant overhead**: only addition, subtraction, multiplication, and division — no exponents, no iteration, no external calls — so Gas is extremely low (see the Gas comparison in the appendix).

> For the fuller derivation (the triple-equivalence proof across CPMM / Weighted / StableSwap), see Appendices A–D of the technical-edition whitepaper.

3.3 Two Extremely Important Benefits

Benefit 1: Direct swaps between any two goods

Whether you want "bread for milk," "eggs for soybeans," or "shampoo for a screwdriver," to the mall it's the same two-step action — **you don't need 4,950 shops on a commercial street; one central checkout handles all 4,950 kinds of swap demand.**

Benefit 2: Liquidity aggregates naturally

When a supplier restocks the "bread" shelf, that bread's "purchasing power" can be shared by **all customers who want to swap something into bread** — it doesn't get locked to a single "bread ↔ milk" specialty shop like on Uniswap.

Comparison	**Traditional DEX (commercial street · N specialty shops)**	**TTSWAP (one mall · one checkout)**
Listing a new good	Someone must open an "X ↔ new good" shop; long-tail goods go ignored	Put it on a shelf and everyone can swap it
Liquidity distribution	Fragmented across shops, each fighting alone	The whole mall shares one checkout; depth aggregates naturally
User routing	Find paths, cross contracts, compute slippage, pay fees multiple times	Always a direct trade
A new good's effect on old users	None (old shops don't know it exists)	Automatically joins the settlement network

3.4 The Math Behind the Checkout (Intuitive Version)

Step 1 and Step 2 are really two simple algebraic formulas, whose characteristics are:

- **Value parity**: the total value of the A you send in always equals the total value of the B you take out;
- **There is slippage**: the more you send, the less B you get per unit good (preventing a whale from sweeping the shelf clean);
- **No exponents, no iteration**: only addition, subtraction, multiplication, and division, so Gas is extremely low.

If you want the full formulas, see Appendix A of the (technical-edition) whitepaper. This whitepaper guarantees: **you can understand how the mall works without reading the formulas.**

4 Listing Goods: Opening New Shelves and Brand Rights

4.1 Two Core Actions

Each good in the mall can only have one shelf. To provide liquidity, there are only **two paths**:

Method	Who uses it	Scenario
`initGood` (open a new shelf + set the custom initial price)	Anyone	Opening a shelf for a Token for the first time
`investGood` (single-token restock)	Anyone	Adding more inventory to an existing shelf

4.2 Opening a New Shelf: `initGood`

To open a brand-new shelf, you tell the mall three things:

1. **What good you're listing** (`T_GoodKey` : type + contract address + sub-id);
2. **How many units you're placing** (`amount1` = initial inventory Q_0);
3. **What initial total value you tag it with** (`amount0` = initial value V_0, i.e. the pricing anchor).

The contract runs 4 checks:

- $V_0 \geq 5 \times 10^{14}$ (about the 500-USDT-equivalent magnitude; too cheap and opening is forbidden, `TTSwapError(35)`);
- $V_0 \leq 2^{109}$ (overflow protection);
- $500{,}000 \leq Q_0 \leq 2^{109}$ (inventory can't be a dust pool, `TTSwapError(36)`);
- This good has **never had a shelf opened before** (`TTSwapError(5)`).

Once it passes, you become this shelf's **brand (Owner)**:

- You automatically receive a **supply certificate** (shares = initial inventory);
- You **don't automatically mine TTS** — opening a shelf itself doesn't stake; TTS hash power is only recorded when you later `investGood` and the shelf is flagged as "commitment-type (isPromised)";
- All customers can immediately buy your new good through the checkout using hard currency like USDT / ETH.

Example: The Full Flow of Listing a New Good

Suppose Alice wants to launch a new brand called `$NEW`:

- She mints 100 `$NEW`;
- She believes the fair price is roughly 1 USDT / unit (i.e. 100 USDT of equivalent value);
- She calls `initGood(NEW, packed(V₀=100 USDT equivalent, Q₀=100 units), ...)`.

Once done:

- The `$NEW` shelf is born: `V = 100 USDT`, `Q = 100 units`, `I = 100 units`;
- Alice holds the initial supply certificate: `shares = 100`;
- Any customer holding USDT can call `buyGood(USDT, NEW, ...)` to swap directly for `$NEW`.

Key point: Alice **only prepared 100 `$NEW`** — she prepared no USDT counterpart asset at all.

Rights for the First to Open a Good's Shelf (First-Mover Operating Rights)

TTSWAP treats "being the first to open a shelf for a given ERC20 in the mall" as **a commercial pioneering act** — it means the shelf setup, community cold-start, brand relationship management, and early-buyer education are all borne by the first shopkeeper. To incentivize this work, the protocol grants the first mover a set of **time-limited operating rights**:

- **Operating term**: the ecosystem recommends a **2-year default** (the contract doesn't hard-code an expiry; it relies on `changeGoodOwner` for market-driven handover);
- **Operating income**: during this period, the **operator commission (operatorFee)** from this shelf's withdrawal-profit split belongs to the brand, typically **1%–3%**;
- **Payout method**: commissions accumulate in `commission[owner]` and can be claimed anytime via `collectCommission`;
- **Extended privileges**: the brand can use `modifyGoodByGoodOwner` to adjust fee rates, amplification factor, and withdrawal slice; and can use `lockGood` for an emergency freeze.

Design motivation: give the first shopkeeper a clear, predictable economic return in the first 2 years, avoiding "doing the work only for someone else to reap." After 2 years, the brand identity can transition to a more capable successor via an **operating-rights auction** as described in §4.5, so the shelf is always run by "whoever is most motivated to run it" over its lifecycle — and you can bid in the auction yourself to obtain the operating rights.

4.3 The Pricing-Anchor Good: Who Plays the "Mall's Dollar"

There is no longer a separate "meta good" pool-creation entry. In practice, **the first stablecoin the community elevates to hard currency (usually USDT)** naturally takes on the pricing-anchor role:

- When a new shelf opens, the founder sets V_0/Q_0 (i.e. the initial price) themselves;
- Later trading, restocking, TTS unlock price checks, etc. all reference the hard-currency shelf's current price via paths like `ishigher(TTS, USDT, ...)`;
- Hard currency is elevated by the **CEO (Market Admin)** via `modifyGoodByAdmin` per community resolution — it is not an automatic promotion.

> **In practice**: a mainnet deployment usually treats USDT as the primary hard currency, but the protocol isn't bound to any single stablecoin — the community can elevate USDC, DAI, etc. via a Class-II proposal.

4.4 The Promotion Path to Hard Currency (Value Good)

If another good wants to become hard currency, the flow is:

1. First open a shelf normally with `initGood` (a normal good by default);
2. After accumulating reputation and depth, the **CEO (Market Admin)** flips on the hard-currency flag by calling `modifyGoodByAdmin` per a Class-II community proposal;
3. After that, this good:
    - Can serve as a "pricing anchor" for other new goods (normal goods can too, but hard currency is more authoritative);
    - Enjoys "no hard inventory constraint" treatment;
    - Its configured commission ratios and amplification factor are usually tuned to better values.

This is a "rule-of-people + reputation" process, not an automatic promotion — designed this way to prevent a freshly-launched good from impersonating "hard currency" to trick out liquidity.

4.5 Upgrading / Relinquishing Brand Identity

The brand can also hand its shelf's brand identity to someone else at any time:

- `changeGoodOwner` (admin-only permission): migrate the brand identity to another address;
- `lockGood` (brand or admin permission): emergency-freeze trading on this shelf.

Note: the brand identity and the supply certificate (shares) are **decoupled** — handing off the brand identity does not transfer the supply certificate you hold.

Operating-Rights Auction: The Market Path for Handing Off a Shelf

After the **2-year first-mover operating term** mentioned in §4.2 expires (or when the incumbent brand voluntarily exits early), the shelf's **operating rights** can be handed to the next operator via an auction, so the shelf is run by "whoever bids highest and is most willing to invest in it" throughout its lifecycle.

Why an auction instead of a direct transfer?

Because a shelf's operating rights aren't a static title — they're **the discounted stream of future trading-fee commissions**. Different candidates may value the same shelf very differently (depending on the community, channels, and marketing power they can bring), and a market auction is the fairest way to price it.

Auction flow and fund allocation

1. The auction runs off-chain / on a dedicated frontend (this whitepaper doesn't mandate the implementation; the community can choose an English auction, Dutch auction, or sealed bids), and the item being auctioned is this shelf's **brand identity**;
2. The final auction proceeds are **automatically split into two parts** per protocol rules:
    - **70% to the original operator** (the incumbent brand) — rewarding their investment in early construction, community operations, and early-buyer education;
    - **30% to the mall protocol** (deposited into the `commission[address(0)]` community treasury) — used for subsequent buyback-and-burn / operating budget / ecosystem incentives (the specific use decided by a Class-III community proposal);
3. Once the split is confirmed and funds arrive, the **admin (Market Manager)** initiates `changeGoodOwner(goodid, new brand address)` to migrate the shelf identity;
4. The new brand enjoys `operatorFee` commissions from the moment the migration completes, and regains brand privileges like `modifyGoodByGoodOwner` / `lockGood`.

Boundaries and protections

- The auction **only transfers the operating rights (owner identity)** — it never touches any supplier's supply certificate (`proofs`) or TTS hash power;
- If the auction fails, the original operator keeps running the shelf (operating rights are not forcibly stripped);
- `changeGoodOwner` is executed with admin permission, meaning every handover enters the on-chain event log for easy community auditing;
- The 70 / 30 split of auction proceeds is a **protocol-level recommendation**; the specific implementation (escrow contract, deposit, objection period, etc.) will iterate with community

governance.

Intuition: just like when a physical shop's lease expires and the landlord re-lets it publicly for the next shopkeeper to take over. TTSWAP moves this on-chain, letting a shelf's operating rights flow freely in the market while safeguarding the original operator's deserved return and binding the protocol's own interests to the shelf's long-term performance.

5 Supplier Restocking: Why One Good Alone Is Enough

5.1 Restocking with Just One Good — Does That Make Sense?

In a traditional DEX, restocking often requires preparing two goods of equal value at the same time. TTSWAP is closer to a real mall: **you can restock even bringing just one kind of good.**

Just call `investGood(goodKey, packed(0, quantity), ...)` — **everything is auto-priced at the shelf's current price.**

5.2 Single-Token Restocking Rules

Scenario	Caller	Notes
Restock at the shelf's current price	Any user	The protocol converts your batch's value at `V/Q`; if the declared price is higher than the pool price it reverts (`TTSwapError(47)`)
Minimum value	Any user	A single restock's converted value must be ≥ 10^{12} (`TTSwapError(38)`)

5.3 The 6-Step Flow of a Single-Token Restock

Using Bob restocking the `$NEW` shelf as an example:

1. **The checkout first reads the shelf state**: it reads current V, Q, I;
2. **Convert at the current price**: after deducting the investment fee, compute this restock's contributed value at the pool price;

3. **Read the amplification factor** (`enpower`): dynamically scaled by pool depth, capped by the `power` config;
4. **Virtual amplification**: the real stock-in is recorded into the virtual display depth at the amplification factor;
5. **Compute supply-certificate shares**: mint Bob a certificate pro rata to his share;
6. **Write back to the shelf** + **conditional TTS accounting**: only when the shelf is flagged `isPromised` does `stake` record TTS hash power.

Price neutrality: before and after the restock, the formulas guarantee the pool's unit pricing won't shift dramatically from a single restock.

5.4 Why a "Single-Token Restock" Can't Siphon Off Other Goods

This is what many readers care about most: **if you can restock with just one kind of good, can't an arbitrageur "cram in a normal good and siphon out the USDT"?**

The answer is: **not easily.** The protocol uses the **safeLine** to limit a single trade's impact on shelf depth — when sell pressure pushes inventory to the upper bound, or buying pulls inventory to the lower bound, the checkout refuses it (`TTSwapError(55/56)`).

5.5 A Very Intuitive Real-World Comparison

> A supplier brings 1000 bottles of mineral water to the community mall — actually delivering 200 bottles, with 800 bottles virtually displayed, at a stock-in price of 10 yuan/bottle.
> So this shelf holds at most 1000 bottles (actual + virtual display).
> If no one comes to buy, and other users want to sell 500 bottles to swap for other goods, the mall owner will surely refuse — **the shelf is full; taking in more would let them rot in the warehouse and crowd out other goods' space.**
> Only when consumers actually start buying water, freeing up 200 slots on the shelf, can other users sell 200 units to swap for other goods.

TTSWAP's handling of normal goods is exactly isomorphic to this mall:

Real-world mall	**TTSWAP**
Shelf capacity 1000 bottles	I + virtual display quantity (the LP support ceiling)
Supplier comes to restock	Customer sells a normal good to the shelf (`buyGood A → B`)

Real-world mall	TTSWAP
Consumer comes to buy water	Customer buys a normal good (`buyGood B → A`)
Owner refuses excess	`TTSwapError(55)` / `TTSwapError(56)` (safeLine)
The milk and biscuits on the shelf	The other goods in the mall (USDT, ETH, etc.)

6 Customer Checkout: Two Ways to Buy

Customers in the TTSWAP mall have two ways to place an order:

Method	Term	Scenario
`buyGood` (specify "how much you pay")	Exact-In	"I have 100 bucks — how much bread can I buy?"
`payGood` (specify "how much you get back")	Exact-Out	"I want exactly 10 loaves of bread — tell me how much to pay"

Both order types support:

- **Direct swaps between any two goods** (good A → B, no need to open a separate pair shop for them);
- **On-chain deadline protection** (auto-void on expiry, preventing stale signatures);
- **Min / max limits** (preventing slippage abuse);
- **Merchant mode** (see §7, X402).

6.1 `buyGood` : I Have X, Seeking Y

Narrative: You walk into the mall with 100 USDT wanting to swap for ETH. You tell the checkout:

> "I'll give you 100 USDT — however much ETH is fine (but no less than 0.025 ETH, otherwise forget it)."

The contract executes:

1. The checkout computes purchasing power ΔV from the 100 USDT;
2. It uses ΔV against the ETH shelf to compute how much ETH you get;
3. It verifies it's $\geq$ your minimum floor (slippage protection), reverting if not met;

4. It deducts 100 USDT from your account and transfers you the corresponding ETH;
5. It deducts a small trading fee, automatically recorded into the six-party fee-split pool (see §10).

Key fields:

- `amount0` = the amount of USDT you pay = 100 (fixed);
- `amount1` = the minimum ETH you'll accept = 0.025 (slippage floor);
- `deadline` = the cutoff timestamp (strongly recommended to be non-zero).

6.2 `payGood` : I Want Exactly Y Back

Narrative: You want to buy exactly 10 loaves of bread as a gift — not one more, not one less. You tell the checkout:

> "Give me exactly 10 loaves; I'm willing to pay no more than 50 USDT."

The contract executes:

1. The checkout works backward: how much ΔV do 10 loaves need?
2. Works backward again: to top up that much ΔV, how much USDT do you pay?
3. If the USDT amount exceeds your set ceiling (50), it reverts;
4. It deducts the corresponding USDT from your account and transfers you exactly 10 loaves.

Key fields:

- `amount1` = the exact number of loaves you want = 10 (fixed);
- `amount0` = the maximum USDT you're willing to pay = 50 (slippage ceiling);
- `external_info.amount0` = can carry business info (e.g. a payment order number);
- `external_info.amount1` = the cutoff timestamp (strongly recommended to be non-zero).

6.3 `payGood` 's "Same-Token Fast Path": Dedicated to Merchant Collection

There's a very common scenario:

> **Xiaoming wants to pay a shop 10 USDT for an order using USDT, and the shop supports TTSWAP collection.**

This isn't a "swap" at all — it's a **pure transfer.** TTSWAP optimizes for this case:

- When `_goodid1 == _goodid2` (paying and receiving the same good), the protocol **skips AMM computation entirely**;

- It deducts 10 USDT from Xiaoming's account → transfers it straight into the shop's account;
- It only deducts a fixed **execution fee** `executeFee` (about 0.05 USDT, extremely low);
- No slippage, no sandwich risk, no pricing uncertainty.

Comparison: On Uniswap, to pay USDT to a merchant you'd either loop through USDT → USDC → USDT (paying slippage twice) or route through an extra escrow contract. TTSWAP's same-token fast path is **natively one step.**

6.4 Comparison of the Two Trading Methods

Scenario	**Recommended function**	**Who's specified**
Ordinary swap (I have 100, get as much as possible)	`buyGood`	Specifies input
Exact collection (merchant collects 10 USDT)	`payGood` (same-token fast path)	Specifies output
Exact payment (I want exactly 10 ETH, pay whatever)	`payGood` (cross-token)	Specifies output
Order system (a Web2 bill is generated then paid)	`payGood + external_info`	Specifies output

7 Merchants and Remote Payment (X402)

7.1 The Problem: What Does a Customer Need to Place Their Own Order?

In ordinary on-chain payment, you as the customer must yourself:

1. Have the transaction fee ready (Gas, possibly ETH);
2. Connect a wallet, sign, and send the transaction;
3. Wait for on-chain confirmation.

This is very unfriendly to **ordinary users who have never touched crypto**: they have no ETH, don't understand Gas, and can't use a wallet.

7.2 The Solution: X402 Merchant Mode

TTSWAP has built-in support for the **X402 protocol**. The idea is very simple:

> **The customer only signs an order — no need to go on-chain; the "merchant" submits the order on-chain on the customer's behalf and earns a fixed execution fee.**

Roles:

- **Customer (Trader)**: signs a standard EIP-712 order "here's what I want to buy" on their phone, paying no Gas;
- **Merchant (Executor)**: receives the customer's order, verifies the signature, and pays the Gas themselves to submit it on-chain;
- **Mall (TTSWAP)**: executes the order + **automatically deducts a fixed execution fee** from the result for the merchant.

The execution fee is called `executeFee`, the contract constant `50_000_000_000` (5×10^{10}).

> **Exact figure**: at USDT (6-decimal) precision ≈ **0.05 USDT** (half-a-cent magnitude). Nearly zero to the customer, but enough to sustain merchant operations at volume.

7.3 Order Format (EIP-712)

The customer signs one of two order types:

Buy order (corresponds to `buyGood`):

```
buyGood(
    address _trader,      // customer address
    address referral,     // referral (optional)
    uint256 _goodid1,     // what to pay (T_GoodKey.toId())
    uint256 _goodid2,     // what to want
    uint256 _swapQuantity,// how much to pay / minimum to receive
    bytes   data,         // transfer authorization data
    uint256 deadline,     // expiry timestamp
    uint256 nonce         // anti-replay
)
```

Payment order (corresponds to `payGood`):

```
payGood(
    address _trader,
    address recipient,    // recipient address
    uint256 _goodid1,
    uint256 _goodid2,
    uint256 _swapQuantity,
    uint256 external_info,// business metadata (order number + deadline)
    bytes   data,
    uint256 nonce
)
```

7.4 What Merchant Mode Means for Merchants

Scenario: A food-delivery merchant's rider holds up a phone showing a TTSWAP collection QR code; the customer just scans and signs — no ETH, no Gas fee needed.

- The rider (Executor) picks up the order and submits it on-chain;
- The customer (Trader) only signs the payment intent, bearing no Gas;
- The rider earns: `executeFee` (about 0.05 USDT / order) + economies of scale from aggregating many orders;
- The merchant (Recipient) receives exactly `_swapQuantity`.

This gives **TTSWAP a Web2-grade "scan-to-pay" experience.**

Merchants Only Care About Their Collection Token, Not What the Customer Pays With

Traditional on-chain collection has a hidden barrier: **the merchant has to tell the customer "please pay with token XX."** If the customer only has ETH and the merchant only accepts USDT, a single payment gets stuck on the roundabout path of "find a swap entry → do a swap → then pay." TTSWAP's `payGood` pulls this inside the protocol from the very start of its design:

- **The merchant does only one thing**: fill in their **desired collection token** **`_goodid2`** and the **exact received amount** **`_swapQuantity.amount1`** in the collection QR code / order;
- **The customer pays with any good they hold**: when signing, the customer passes in **any ERC20 (or native ETH)** **`_goodid1`** they're willing to spend from their wallet — it can be USDT, ETH, WBTC, or any listed long-tail token;
- **The protocol completes "auto-swap + exact receipt" in one call**: the contract works out how much `_goodid1` to spend via the exact-out back-calculation in §6.2, swaps the customer's token for what the merchant wants at the central checkout within a single transaction, then

transfers the exact amount to the merchant's account — the process is fully transparent to the merchant;
- **Slippage protection remains**: the customer sets `amount0` in the order as the maximum they'll pay, and if the exchange rate exceeds this ceiling it reverts directly — no "passive order-filling."

Mall-scenario intuition: the merchant prices the good as "I want to collect 10 USDT," and the customer hands over "a membership card worth 1 ETH" or "the balance of some long-tail token" — the checkout internally swaps the membership card into USDT before giving it to the merchant, who only ever sees 10 USDT arrive.

Three benefits for merchants:

1. **Unified revenue accounting**: no matter what the customer pays with, what flows into the books is always the merchant's chosen collection token (usually a stablecoin / their own token), giving clean financial accounting;
2. **Covers all token-holding customers**: the merchant's potential customer base = every token-holding user in the entire TTSWAP mall, not just "the small subset holding USDT";
3. **No need to build a swap entry**: traditionally, integrating stablecoin payment requires extra escrow, swap routing, and slippage monitoring — TTSWAP does all three natively in the contract.

Special Case: Same-Token Fast Path (merchant's token = customer's payment token)

When the customer happens to already hold the token the merchant wants (`_goodid1 == _goodid2`), the protocol skips the AMM and takes the "same-token fast path" from §6.3:

- The protocol does no swap and charges no slippage;
- It only deducts `executeFee` (about 0.05 USDT) as the merchant's relay fee;
- Both merchant and customer enjoy the same Web2-grade "scan-to-collect" experience.

7.5 Small Orders Signed in Real Time / Large Orders Settled On-Chain: Face-to-Face, Second-Level Trading

X402 opens another door: **the same EIP-712 signature can be both a payment commitment "valid right now" and a settlement certificate "batch-submitted on-chain later."** Merchants can process by order amount tier, replicating brick-and-mortar card-swipe "PIN-free small vs. PIN-required large" risk tiering — **day-to-day face-to-face trades no longer wait 10–30 seconds for on-chain confirmation.**

7.5.1 Tiering Strategy (merchant-defined threshold T)

Tier	Amount range	Merchant action	Customer experience
Small order	$\leq T$ (e.g. ≤ 10 USDT)	Verify signature locally → deliver / provide service immediately; enqueue the signature into a **pending-settlement queue** for batch on-chain submission later / before closing	Scan → cleared in 1 second, **no waiting for block confirmation**
Large order	$> T$	Submit the signature on-chain → wait for successful confirmation → then deliver	Scan → wait 10–30 seconds → cleared

The threshold T is set by the merchant themselves, analogous to a card's "**contactless PIN-free limit**" — the same merchant can even set different T values for different product categories, e.g. a café at T=20 USDT and electronics at T=500 USDT.

7.5.2 Why Small Orders Can "Clear First, Submit On-Chain Later"

For the merchant, **the customer's signed EIP-712 order is itself a certificate redeemable on-chain**, like a check with a PIN:

- **The signature is a cryptographic receipt**: it contains fields like the customer address, payment amount, recipient, nonce, and deadline — impossible to forge or tamper with;
- **The on-chain timing is controlled by the merchant**: as long as it's before the `deadline`, the customer's nonce hasn't been used, and the account balance and authorization are still in place, the merchant can redeem it **at any time**;
- **You can prove it even if it fails**: the signature itself is evidence, usable for off-chain arbitration or the §11.3 board dispute process;
- **Fees are cheaper**: batch submission (combined `multicall`) saves Gas versus one transaction per order.

7.5.3 Risk Boundaries and Hedging Measures

Instant clearing of small orders **does carry a small-amount default risk** (the customer moves the funds or revokes authorization before the merchant submits on-chain), but it's converged to extremely low by three measures:

1. **Amount ceiling**: T is the merchant's maximum single-transaction loss exposure — controllable like a card's "PIN-free limit";

2. **`deadline` auto-void on timeout**: the customer's signed order carries an expiry; the merchant must submit on-chain before it expires or it becomes void;
3. **Auditable customer `cancelNonce()`**: if a customer frequently cancels un-submitted signatures, the merchant can simply remove that address from the "small-order PIN-free" list (like a card credit-limit reduction).

For extremely risk-averse merchants, they can **always set T to 0** (all orders wait for on-chain), falling back to default behavior; for merchants chasing extreme speed (convenience stores / cafés / taxis), they can open T up to "the ceiling of everyday ticket size" and enjoy an off-chain-grade experience.

7.5.4 A Typical Storefront Scenario

> **Scenario: a corner café, priced at 3 USDT/cup.**
>
> 1. The customer scans the code; the phone wallet pops up the order: `pay 3 USDT to CoffeeShop`;
> 2. The customer signs with their fingerprint → **the phone sends the signature to the merchant's POS (BLE / NFC / QR code)**;
> 3. The POS runs `ecrecover` locally + checks whether the signature fields match the order (**done in about 100 ms**);
> 4. The POS shows "✅ Payment successful" → the clerk hands over the coffee → the customer leaves;
> 5. **Before closing**, the merchant bundles all the day's signatures and `multicall`s them on-chain in one shot; `executeFee` is deducted only once and Gas is amortized to nearly zero per order.

The whole customer flow is **≤ 2 seconds** — matching the experience of offline card swipes / scan payments — and **chain confirmation speed is no longer the bottleneck for face-to-face trades.**

7.5.5 Summary: Three Amount Tiers × Three Trust Tiers

Stacking the capabilities of §6 / §7.3 / §7.4 / §7.5, TTSWAP merchant payment can cover every scenario from "small coffee purchases" to "large institutional amounts":

Amount tier	Path	Latency	Applicable scenario
Micro amount ($\leq T_1$, e.g. ≤ 10 USDT)	Small-order real-time signature (§7.5) + same-token fast path (§6.3)	**~1 second**	Convenience store / coffee / ride-hailing

Amount tier	Path	Latency	Applicable scenario
Medium amount ($T_1 \sim T_2$)	Small-order real-time signature + AMM swap (§7.4)	**~2–6 seconds, the on-chain execution success time**	Dining / daily goods
Large amount ($> T_2$, e.g. > 1000 USDT)	Large-order on-chain settlement	**on-chain confirmation time**	Electronics / jewelry / B2B settlement

By adjusting the two thresholds T_1 / T_2, the merchant can freely trade off between "ultra-fast experience" and "zero default risk" — this is a **risk-control knob** the TTSWAP protocol natively hands to merchants.

7.6 Why Won't the Merchant Act Maliciously?

You might worry: will the merchant deliberately tamper with your order when submitting it on-chain? The answer is **they can't**:

- Every order is bound to its contents by the customer's **EIP-712 signature**;
- The contract **verifies the signature with `ecrecover` before execution**; any tampering invalidates the signature;
- The signature contains the customer's incrementing `nonce`, and each signature can be used only once;
- The optional `deadline` lets the signature auto-expire;
- The customer can proactively void all **un-submitted** signatures by calling `cancelNonce()`.

7.7 Comparison with Uniswap / 1inch Fusion

Experience point	Uniswap	1inch Fusion	TTSWAP X402
Customer needs Gas	✅ Yes	❌ No	❌ **No**
Customer needs a wallet	✅ Yes	✅ Yes	✅ **Yes** (but signature-only)
Supports any good → any good	⚠️ Must open a pair shop	✅ Aggregated	✅ **Natively one store**

Experience point	Uniswap	1inch Fusion	TTSWAP X402
Exact same-token collection	❌ Needs a detour	❌ Needs a detour	✅ **Native same-token fast path**
Relay fee	—	Floating	**Fixed executeFee**

8 The Shelf Inventory "Seatbelt" (safeLine)

The mall's most core, yet most easily overlooked, safety design is:

> **Each shelf's tradeable depth is boxed into a healthy range by the `safeLine` upper/lower bounds.**

It replaces v1's `Q ≤ I` hard constraint with two 8-bit config fields (upper bound / lower bound). A default value of 0 means "no extra tightening"; the admin can set a tighter range for long-tail shelves.

8.1 Why This Seatbelt Is Needed

First imagine a mall **without** the seatbelt:

> A scammer registers an "air token" `$AIR` and tells the mall:
> "I'll put 10,000,000 `$AIR` on your shelf, initial value 100,000 USDT equivalent."
> Then he starts selling `$AIR` to the mall, taking away the mall's real USDT / ETH.
> Because no one actually wants to buy `$AIR`, but the mall still obediently hands him USDT / ETH.
> **Result**: the mall's USDT / ETH is drained by the air token, and all suppliers get burned.

This is the classic "reverse drain via a normal good" attack — without constraints, it's bound to happen.

8.2 How TTSWAP Sets This Seatbelt

After each large swap's sliced computation ends, the contract checks:

- **Sell-in side** (customer "sells" a good to the shelf): current inventory Q must not exceed the upper bound → otherwise `TTSwapError(55)`;
- **Buy-out side** (customer buys a good from the shelf): current inventory Q must not fall below the lower bound → otherwise `TTSwapError(56)`.

The upper/lower bounds are computed proportionally from the `safeLine` config and the baseline inventory `I + virtual quantity`. The admin can customize the tightness for different shelves via `modifyGoodByManager`.

8.3 The Three Behavioral Patterns This Seatbelt Brings

Pattern 1: Large sell pressure gets refused upon hitting the upper bound and must wait for buying or withdrawals to free up space.

Pattern 2: A newly-listed good needs real demand to digest inventory before it can keep absorbing sell orders.

Pattern 3: When the external price crashes, dumping can't drain the shelf in one go — subsequent orders hit the wall, and arbitrage plus LP withdrawals gradually repair the price.

8.4 Normal Good vs. Hard Currency: The Essential Difference Between the Two Treatments

Scenario	Normal Good	Hard Currency
Large sell pressure hits the wall?	**Yes** (hits the safeLine upper bound)	Usually a wider range, configured by the admin
Single-side restock pricing	Unified `investGood` in-pool price	Same as left
Can serve as a pricing anchor	Can, but not authoritative	More authoritative after community elevation

Pattern: normal goods are stricter on both "restock pricing" and "sell-out ceiling," to protect suppliers from dilution by malicious goods. Hard currency's privileges correspond to its identity credit, which passed "Class-II community proposal approval."

8.5 Triple Defense: Resisting Impermanent Loss

Beyond the "inventory seatbelt," TTSWAP has three layers of defense to protect supplier principal:

1. **First line of defense: protocol-level value anchoring (code layer)**
   The supply certificate records the **historical stock-in quantity**, and withdrawal returns at this historical ratio — **independent of the current price.** Even if the shelf price swings wildly, the quantity of goods the supplier reclaims is deterministic.

2. **Second line of defense: brand subordinated buffer (capital layer)**
   The initial liquidity of hard currency / brands is contributed by the issuer, and it **bears value shortfalls first under extreme conditions**, acting like the "subordinated tranche" of insurance.
3. **Third line of defense: TTS governance compensation fund (governance layer)**
   For losses that can't be covered in extreme cases, the DAO resolves to launch a TTS auction / release mechanism to compensate affected users.

Intuition: an individual supplier doesn't bear IL risk alone — their funds are safeguarded under three layers of defense. Typically, compounded fees + compensation are enough to converge this risk to an acceptable level.

9 Supplier Settlement: How to Reclaim Principal and Profit

9.1 The Core Rules of Withdrawal

When a supplier wants to withdraw (reclaim principal + profit), they call `disinvestProof(proofid, shares, gate, trader, signature)`:

1. The contract verifies certificate ownership (it must be initiated by you yourself);
2. **Return pro rata to the historical stock-in quantity on the certificate**:
   - The share you're withdrawing this time is `X%` of the certificate's total shares;
   - Return `X%` of the **originally stocked-in quantity** to you;
   - **Independent of the current shelf price**;
3. Compute the accumulated "profit" (current shelf net value – historical net value), distributed via the six-party split among supplier / brand / storefront / referral / customer / mall;
4. The withdrawal fee (`disinvestFee`) is deducted proportionally and stays in the shelf (effectively raising the per-share net value for other suppliers — an indirect compounding);
5. Sync TTS unstaking: only when this certificate has ever recorded TTS hash power (on an `isPromised` shelf) and this withdrawal's corresponding `disinvestTTSValue > 0` does it `unstake` to mint TTS.

9.2 Why "Return by Historical Quantity" Achieves Zero Impermanent Loss

Traditional AMMs (Uniswap V2/V3) have a classic problem: **when a supplier withdraws, the two-token quantities they receive get deformed by intervening price changes.** For example, you deposit 1 ETH + 2000 USDT (value 4000 USDT), and when ETH rises to 3000 you withdraw, possibly getting 0.8 ETH + 2400 USDT (value 4800 USDT). **The value went up, but your ETH quantity went down** — that's impermanent loss.

TTSWAP's approach:

- You deposit 10,000 `$NEW`, and the certificate records "historical stock-in = 10,000";
- No matter how the shelf price swings, on withdrawal the protocol returns 10,000 by the certificate;
- The fees, welfare injections, etc. accumulated in the meantime are given to you additionally as a **net-value increment.**

Result: the **quantity of goods** a supplier receives on withdrawal doesn't shrink but grows (principal + profit) — this is the protocol-level implementation of "zero impermanent loss."

9.3 Withdrawal Slices (Disinvest Chips)

If a whale withdraws too much at once, it impacts other suppliers and customers. The protocol sets a **withdrawal-slice limit**:

- Every shelf has a config item `getDisinvestChips`;
- A single withdrawal's quantity / value can't exceed the slice cap;
- Exceeding it reverts with `TTSwapError(26)` or `TTSwapError(27)`.

Benefit: it smooths a withdrawal's impact on shelf depth, preventing a whale from dumping and dragging down other suppliers.

9.4 A Complete Withdrawal Example

Suppose Bob holds a supply certificate:

- `shares = 10000`, `invest = 10000 $NEW`;
- Trading fees accumulated on the shelf in the meantime;
- Bob wants to withdraw 50% (5000 shares).

The protocol executes:

1. Return pro rata to history: `actualReturn = 10000 × 5000 / 10000 = 5000 $NEW` ;
2. Compute the value corresponding to the current net value;
3. Net profit = current net value – historical principal ≈ some positive number Δ;
4. Deduct the withdrawal fee `disinvestFee` ;
5. Six-party split (Bob usually collects `liquidFee + customerFee` two tiers, plus principal);
6. In Bob's account: **5000 `$NEW` + Δ profit share + returned TTS hash power**;
7. Bob's certificate updates to the remaining `shares = 5000, invest = 5000` .

9.5 Permission Distribution During Withdrawal

`disinvestProof` only allows **the proof's owner in person** to call it:

- The protocol verifies via `S_ProofKey(trader, currentgood).toId() == proofid` ;
- Even with someone else's signature you can't withdraw on their behalf (**there's no delegated-withdrawal mechanism**);
- This is intentional — withdrawal involves fund returns and has a large risk surface, so outsiders are forbidden from operating on your behalf.

9.6 Mining TTS While Supplying: A Second Income for Contributing to the Mall

Suppliers in TTSWAP don't just earn trading-fee commissions — they also **mine TTS tokens simultaneously.** This income line aligns with "**however much value you contribute to the mall, the mall distributes you that much TTS token accordingly.**"

9.6.1 When Mining Happens

Action	**Records TTS hash power?**
`initGood` opening a shelf	**No**
`investGood` restocking	**Only when the shelf is flagged `isPromised`**
`disinvestProof` withdrawing	Returns hash power by the withdrawn share (via `unstake` if present)

Each qualifying `investGood` automatically runs `L_Proof.stake(TTS, msg.sender, investvalue)` when writing back to the shelf — **completed within the same transaction**, with no separate `claim` needed.

9.6.2 TTS Hash Power Is Accounted by "Contribution Value"

The TTS mining measure is fully decoupled from trading fees:

- **Value, not unit count**: restocking 10,000 long-tail tokens vs. 10,000 USDT differs hugely in hash power — what counts is `investvalue`;
- **Commitment-type shelves**: Launchpad / IDO-type projects have the admin set `isPromised`, and only then does restocking count as mining;
- **The larger the value, the more hash power**: restocking hard currency is usually the most efficient.

9.6.3 Daily Emission Pool: 200 Million Total / ~50-Year Linear Release

All LPs share one on-chain mining pool. Every 86,400 seconds (1 day) the protocol auto-settles once:

$$\text{Daily emission} = \frac{200\text{M TTS} - \text{current circulating supply}}{18{,}250} \times \frac{\text{chainRatio}}{10{,}000}$$

(If the settlement result is below `1e12`, a floor of **1 TTS** applies.)

The emitted TTS is distributed by each active LP's **hash-power share**. If your hash power is `X%` of the pool, you get `X%` of that day's emission. **There's no "miner fee," no "subsidy expiry" — this release curve is built into the contract.**

9.6.4 When TTS Mining Rewards Are Realized

Action	TTS hash power	Do you get TTS tokens now?
Restock `investGood` (isPromised)	**Hash power recorded immediately**	No (awaits daily-emission settlement)
Continuously holding the supply certificate	Hash power participates in emission daily, pro rata	No (rewards accumulate on the certificate)
Withdraw `disinvestProof`	**Returns hash power pro rata** to the withdrawn share	**Yes**, `L_Proof.unstake` mints the corresponding TTS rewards to your account

So Step 5 of §9.1's withdrawal, "sync-return your corresponding hash-power share in the TTS token pool," actually **also delivers the TTS rewards accumulated during your holding period.** The more shares you withdraw, the more of the hash power's corresponding TTS is realized.

> If you only withdraw part, the remaining shares keep mining; if you withdraw everything, hash power resets to zero — equivalent to "exiting the mining pool and settling the final batch of rewards."

9.6.5 Three Incomes Stacked: The Supplier's Economic Ledger

Counting trading-fee commissions (§10), zero-impermanent-loss principal return (§9.2), and TTS mining, a TTSWAP supplier collects three returns at once:

Income source	When it arrives	Denomination
① Principal	Returned by historical quantity on withdrawal	The good you supplied itself
② Trading-fee commissions (`liquidFee + customerFee`)	On withdrawal or via `collectCommission`	The good of the corresponding shelf
③ TTS mining rewards	Auto-minted via `unstake` on withdrawal	**TTS**

Intuition: every second you provide liquidity to the mall, you're collecting three kinds of "interest" simultaneously — compounding on your spot principal, cash flow from trading commissions, and TTS equity representing ownership of the mall. The first two are tied to the shelf itself; the third is tied to the entire mall's long-term value.

10 The Six-Party Fee Split: Who Gets a Cut of Each Trade

10.1 The Six-Party Split: The "Whoever Contributes, Benefits" Principle — the Economic Foundation of the Mall Ecosystem

The profit generated by every "withdrawal" or "trade" is split among six parties:

Role	Identity in the mall	Typical share	Notes
Supplier (LP)	Whoever restocks the shelf	**50% – 80%**	The largest share, rewarding risk-takers

Role	Identity in the mall	Typical share	Notes
Brand (Operator)	This good's issuer / owner	1% – 3%	Whoever lists it benefits
Storefront (Gate)	The frontend / aggregator displaying this good	5% – 25%	Whoever drives traffic benefits
Referral	Whoever brings the customer in	5% – 10%	Acquisition reward
Customer	The customer themselves (usually a "discount rebate")	5% – 10%	Retention reward
Mall (Platform)	The mall itself (community treasury)	2% – 8%	Operations + buyback-and-burn

10.2 These Six Parties Are TTSWAP's Innovation

Compared with traditional DEXs:

Protocol	Fee recipients	Notes
Uniswap V2	100% LP	One party takes all
Uniswap V3	LP + Protocol	Two parties
Curve	LP + veCRV holder	Two parties
TTSWAP	**6 parties**	**Protocol-native**

Core design philosophy: TTSWAP writes "frontend storefront," "referral," and "customer discount" **into the protocol contract**, letting things traditionally done by centralized channels / aggregators / marketing teams be automatically distributed directly by the on-chain contract.

This makes TTSWAP naturally suited for:

- **B2B2C distribution**: when various SaaS integrate TTSWAP, they can earn on-chain commissions via the Gate role;
- **KOL acquisition**: the Referral relationship chain makes community promotion legal and compliant;
- **User loyalty**: Customer discounts automatically return to the user themselves, forming a retention flywheel.

10.3 The Six Parties Must Sum to 100%

There's a hard check in the contract:

```
function checkGoodConfig(uint256 config) internal pure returns (bool) {
    // liquid + operator + gate + referal + cust + platform
    return (liquid + operator + gate + referal + cust + platform == 100);
}
```

Each shelf's six-party shares are configured in the `goodConfig` bits, and they **must strictly sum to 100%.**

This guarantees every profit is fully distributed — no "missing accounts" or "black-box skimming."

10.4 Two Modes of the Split Path

Mode A: No referral (most cold-start scenarios)

- Supplier: `liquidFee + principal return`
- Storefront: `operatorFee + customerFee`
- Mall: `profit − liquidFee + platformFee`

Mode B: With referral (typical active user)

- Brand: `operatorFee`
- Storefront: `gateFee`
- Referral: `referFee`
- Supplier: `liquidFee + customerFee + principal return`
- Mall: `platformFee`

Pattern: once a referral exists, the brand / storefront / referral / customer / mall each get their corresponding share; without a referral, the relevant shares are merged into the storefront or mall.

10.5 Claiming Commissions

All commissions accumulate in `commission[role address]` :

- Any user can call `collectCommission(goodIds[], ...)` to batch-withdraw;
- When the Market Admin calls it, they can also clear `commission[address(0)]` (the community treasury) for the mall to buy back and burn TTS (executed by the CEO per the community budget proposal);

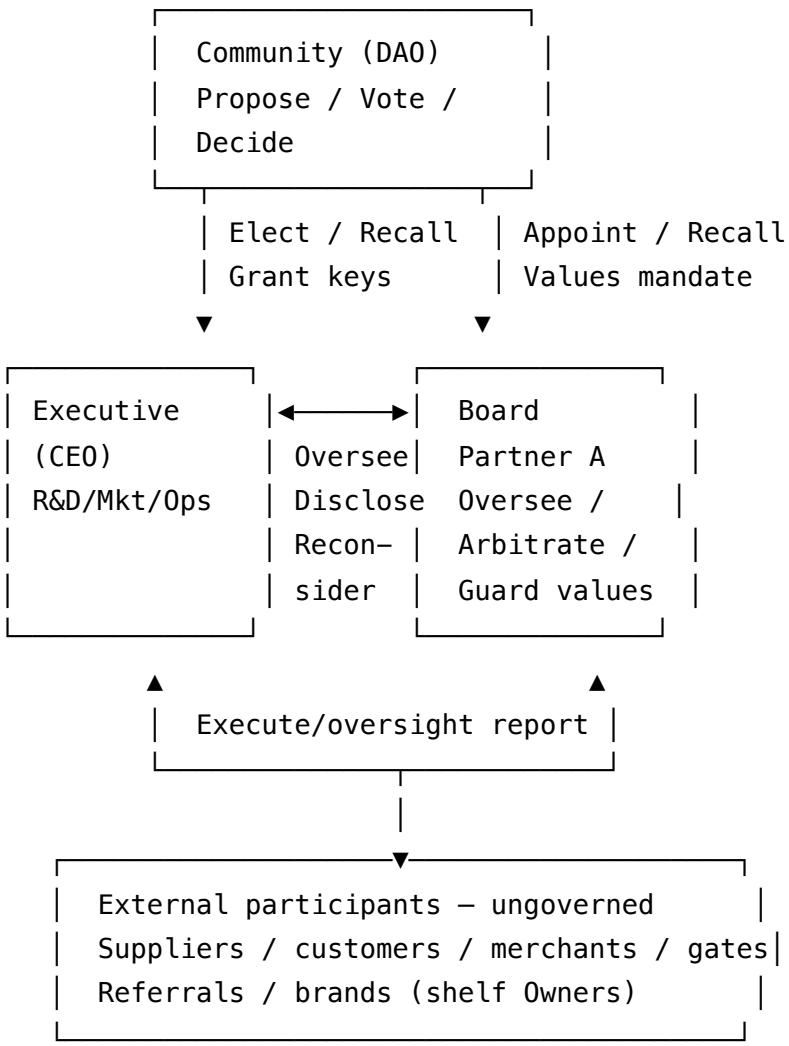
- The contract keeps a 1-unit sentinel (leaving 1 even when emptied) to avoid the high Gas cost of a "cold SSTORE."

11 The Mall's Decision Architecture and Organizational Structure

TTSWAP is a decentralized mall **decided collectively by token holders, executed by a CEO-led team, and its values safeguarded by a board.** It doesn't use a public company's "few directors + multisig + black box" model, but rather a **triangular relationship of community – executive – board**, with the three parties checking each other and full on-chain transparency:

- **Community (Community / DAO)**: the TTS-token-holder community, holding proposal and voting rights — the highest decision layer;
- **Executive (Executive / CEO team)**: a CEO elected by the DAO plus the three business teams (R&D / marketing / operations) they assemble, holding the contract Admin / Manager keys;
- **Board (Board / Partner Type-A holders)**: formed by Partner Type-A TTS holders, responsible for continuous oversight of operations, dispute arbitration, and initiating reconsideration of proposals that violate TTSWAP's core values.

> **Core of the triangular relationship**:



This chapter clarifies three things:

1. **Decision architecture (community / DAO)**: how TTS holders propose → vote → execute;
2. **Organizational structure (executive / CEO team)**: how a CEO is voted in, and how the CEO assembles a team and drives the business;
3. **Board (Board)**: how Partner Type-A holders oversee operations, arbitrate disputes, and guard TTSWAP's core values.

Core principles:

- **Separation of powers**: the community's decision power, the CEO's executive power, and the board's oversight power are mutually independent and check each other;
- **The community is the top layer**: the CEO is elected by the community, and the board is mandated by the community; both the CEO and the board **answer to the community**;
- **The board is independent of the CEO**: the CEO has **no power to appoint or remove** board members; the board holds no contract keys and doesn't participate in daily business;

- **Brands (shelf owners) and external participants are always independent of every governance layer**, unaffected by any governance outcome — any user can withdraw their own funds and claim their own commissions at any time;
- The contract's underlying permission bits (Admin / Manager) are merely **executors of community resolutions**, changing dynamically with voting outcomes.

11.1 Decision Architecture: The TTS Holders' DAO

11.1.1 Governance Principles

- **Anyone can participate**: as long as you hold TTS, you're a "shareholder" of the mall;
- **Weighted by amount**: voting weight is by TTS holdings;
- **On-chain transparency**: all proposals, discussions, votes, and execution records are traceable on-chain;
- **Accountable**: every passed proposal corresponds to one or more on-chain calls, with evidence of who initiated, who voted, and who executed.

11.1.2 Who Can Propose?

> **An address holding a certain amount of TTS token** can initiate a proposal. The exact figure (e.g. 1% of circulating supply / a fixed X tokens / a snapshot threshold) is set by the DAO via the governance contract during deployment and can be adjusted in future proposals.

Why set a threshold:

- Prevent spam proposals — governance with no threshold would drown in "troll-army proposals";
- Give the proposer "skin in the game";
- Keep the threshold at a level **an active community member can reach** (not "only whales can propose").

11.1.3 Who Can Vote? How Is Voting Power Calculated?

- Any address holding TTS **can vote**;
- Voting weight = **your TTS holdings** (whether it's balance-based, lockup-weighted, or snapshot-based tallying is decided by the governance-contract implementation);
- No need to run a node or write code yourself — vote directly via the governance frontend / multisig / governance contract.

11.1.4 The Lifecycle of a Proposal

Every proposal goes through 5 stages:

```
Stage 1: Draft
  The proposer writes the idea clearly on the community forum / Discord and seeks feedb
  Purpose: mature the proposal content and anticipate objections.

Stage 2: Formal Submission (Submit)
  The proposer submits the proposal on-chain, at which point the content is frozen and
  Must meet the proposal threshold (TTS holdings).

Stage 3: Discussion / Vote (Vote)
  Enters a fixed voting period (e.g. 3–7 days, defined by the governance contract).
  All TTS holders vote "for / against / abstain" by their weight.

Stage 4: Tally
  After the voting period ends:
    · Reaches the quorum ✓
    · The for-share meets the requirement for that proposal type ✓
  Both satisfied → the proposal passes.

Stage 5: Execute
  Once passed, the governance contract (or a designated executor) automatically /
  manually calls the corresponding Market / Token contract functions to land the resolu
  Major proposals may need a Timelock cooling-off period before taking effect.

(The board oversees the execution of passed decisions.)
```

11.1.5 Proposal Classification (Four Categories)

TTSWAP's proposals are split into four categories by "scope of impact," corresponding to different passing thresholds:

Category I: Organization & Personnel

Typical scenarios:

- Nominate / recall the **mall CEO**;
- Approve / reject the functional-team leads nominated by the CEO (the heads of R&D, marketing, operations);
- Approve adding or dismantling functional teams;
- In specific cases, grant / revoke the contract's Admin / Manager roles.

Suggested threshold: medium / high (since it directly decides the executive team's leadership).

On-chain expression: once passed, the governance contract calls `setMarketAdmin` / `setTokenAdmin`, etc. to appoint / revoke contract roles.

Category II: Protocol Changes

Typical scenarios:

- Adjust a good's **default fee rate**, **maximum amplification factor M**, **safeLine**;
- Elevate a good to **hard currency (Value Good)**;
- Adjust the **six-party split upper/lower bounds**;
- Upgrade the contract (via `upgrade(newImpl)`);
- Modify global parameters like `chainRatio`.

Suggested threshold: medium (balancing flexibility and stability).

On-chain expression: once passed, the governance contract calls functions like `modifyGoodConfig / modifyGoodByAdmin / setRatio / upgrade`.

Category III: Budget & Finance

Typical scenarios:

- Withdrawal use of public-sale funds (`withdrawPublicSell`);
- Use of the community treasury balance (from `commission[address(0)]`), e.g.:
  - The execution plan to buy back TTS and burn it;
  - Marketing budget appropriation;
  - Developer incentive pool;
  - Partner commission budget;
- A new round of share allocation from the 4C allocation pool (`addShare` / `permitShare` batch airdrop).

Suggested threshold: medium-high (involves real money).

On-chain expression: once passed, the governance contract calls `withdrawPublicSell / collectCommission / addShare / permitShare`, with fund flows transparent and traceable on-chain.

Category IV: Major Matters

Typical scenarios:

- **Trustless endgame** (`disableUpgrade`);
- Emergency **freeze of the entire mall / TTS contract** (`freezeMarket / freezeToken`);
- Emergency **mint / burn of TTS** in special cases (`mint / burn` , only for contract migration / vulnerability fixes);
- Setting or modifying **the governance contract's own parameters** (proposal threshold, voting-period length, quorum, etc.).

Suggested threshold: high (e.g. supermajority + a longer cooling-off / Timelock).

On-chain expression: once passed, executes irreversible on-chain operations — especially `disableUpgrade` , which is a one-time, irreversible action.

11.1.6 Passing Rules (Three Tiers)

Proposal category	Quorum	Passing threshold	Timelock
I Organization & Personnel	Medium	Majority for	Short (e.g. 24–48h)
II Protocol Changes	Medium	Majority for	Medium (e.g. 48–72h)
III Budget & Finance	Medium-high	Majority for	Medium (e.g. 48–72h)
IV Major Matters	High	Supermajority (≥ 2/3)	Long (e.g. 7+ days)

> **The exact figures are set at governance-contract deployment** and can be modified via a Category-IV proposal. This only lists the tiering principles.

11.1.7 Who "Presses the Button" After a Proposal Passes

This is a detail many decentralized projects don't explain clearly: **once a proposal passes, who makes the on-chain call?**

TTSWAP uses two approaches (decided by the governance-contract implementation):

- **Approach A: the governance contract's automatic executor**
  - After a vote passes, the governance contract's `execute(proposalId)` can be called by **anyone** (including bots);
  - The function internally checks the proposal state and whether the Timelock has elapsed, then automatically calls the target contract function;
  - This approach is the most trustless — **whoever presses it, the same thing happens; the logic behind the button is set in code.**
- **Approach B: the CEO as executor**

- After a proposal passes, the CEO or their team is authorized to execute (e.g. CEO multisig);
- Suited to complex proposals needing **off-chain coordination + on-chain multi-step composition**;
- The process is auditable, but with the extra trust that the CEO team won't delay / distort execution.

Core: no matter which approach, **the voting outcome itself is already decided; execution is just "putting the key in the lock."**

11.2 Organizational Structure: The Executive Layer Assembled by the CEO

11.2.1 The Mall CEO (Market CEO)

Identity: the top executive officer elected by the DAO via an **organization-and-personnel proposal.** Answers to the DAO, which can recall them at any time via a new proposal.

Powers and responsibilities:

- **Team-formation power**: nominate the leads of the three functional teams — R&D / marketing / operations (subject to DAO approval or record);
- **Scheduling power**: allocate the three teams' resources, schedules, and priorities;
- **Execution power**: drive the DAO's resolutions (passed proposals) to landing;
- **External representation**: represent the mall in external collaboration (communicating with exchanges, audit firms, partner protocols, regulators, etc.);
- **Budget proposals**: within the budget the DAO authorizes, allocate resources to teams (or initiate finer budget proposals);
- **Emergency response**: when facing a vulnerability / attack / market anomaly, organize personnel to respond immediately and disclose to and seek ratification from the DAO afterward.

Contract-layer expression:

- The CEO holds the `Market Admin` **multisig key** (core good config + Market contract upgrade), the CEO's highest authority on-chain;
- The CEO also holds the `Token Admin` multisig key, for TTS token daily operations (public sale / share allocation / TTS contract upgrade);
- These keys **aren't the CEO's personal property** — they're **"work keys" mandated by the community**;

- The community can revoke these keys at any time via a Category-I proposal (calling `setMarketAdmin(old_ceo, false)` / `setTokenAdmin(old_ceo, false)`).

Three things the CEO cannot do alone:

1. **Cannot press `disableUpgrade` alone** — trustlessness is the authority of the **DAO Admin** key, which is in the **board's** hands, not the CEO's; it must be executed by the board after a Category-IV community proposal passes;
2. **Cannot misappropriate the community treasury alone** — budget use must go through a Category-III proposal;
3. **Cannot adjust their own appointment alone** — organization and personnel must be decided by a community vote.

11.2.2 R&D Team (Tech Team)

The CEO nominates a lead (Tech Lead) and forms the team.

Main responsibilities:

- **Smart-contract development and maintenance**: new features, optimizations, vulnerability fixes;
- **Audit coordination**: liaison with third-party audit firms (OpenZeppelin / Trail of Bits / Spearbit, etc.);
- **Technical upgrade roadmap**: plan new contract versions like V2/V3;
- **SDK / frontend / Subgraph** and other toolchain maintenance;
- **Emergency response**: bug-bounty liaison, vulnerability patches, emergency circuit-breaker coordination.

Contract-layer expression: the Tech Team lead may hold the `Market Manager` multisig key (for daily operational fine-tuning, but contract parameters can't exceed the DAO-limited range); **core upgrades** (`upgrade`) require CEO-level authorization.

11.2.3 Marketing Team (Market / BD / PR)

The CEO nominates a lead (Market Lead) and forms the team.

Main responsibilities:

- **Brand and PR**: the mall's external narrative, media relations, community building;
- **Merchant BD**: recruit new brands to list, and integrate payment / aggregators;

- **Partnership expansion**: strategic partnerships with other protocols / funds / exchanges / wallets;
- **Investor relations**: liaison with institutional investors / VCs / LPs;
- **Content operations**: whitepapers, technical blogs, tutorial videos, offline events.

Contract-layer expression: the marketing team **usually holds no contract Admin / Manager keys**, doing more off-chain work. For matters like "elevating a new good to hard currency," the marketing team initiates the discussion and proposes that the CEO launch a Category-II proposal.

11.2.4 Operations Team (Operations)

The CEO nominates a lead (Ops Lead) and forms the team.

Main responsibilities:

- **Daily operations**: monitor contract operation, Gas costs, abnormal trades;
- **User support**: first-line response on the community / Discord / Telegram;
- **Risk control**: monitor suspicious addresses and coordinate `setBan` bans;
- **Event planning**: execute mining / airdrop / incentive campaigns;
- **Community treasury daily operations**: per DAO budget-proposal authorization, periodically execute `collectCommission` / buyback / burn;
- **Data disclosure**: periodically disclose operational and financial data to the DAO.

Contract-layer expression: the Operations lead may hold `Token Manager` (executing `setBan`) or an **executor node** (executing routine tasks like public-sale fund allocation); sensitive operations still need CEO multisig or DAO authorization.

11.2.5 Mapping Contract Roles to Organizational Positions

The table below maps "on-chain roles" to "organizational positions," so readers clearly see **whose hands the keys are in and where the authorization comes from**:

On-chain role	Usually held by	Authorization source	Typical operations
DAO Admin	**Board multisig**	Category-I community proposal	Permanently lock upgrades (`disableUpgrade`), emergency mint/burn TTS, appoint other Admins
Market Admin	**CEO multisig**	Category-I community proposal	Elevate hard currency (`modifyGoodByAdmin`), Market

On-chain role	Usually held by	Authorization source	Typical operations
			contract upgrade
Token Admin	**CEO multisig**	Category-I community proposal	Public-sale management, share allocation, TTS contract upgrade
Market Manager	R&D / Ops lead	CEO (as Market Admin) authorization	Six-party split adjustment, emergency freeze of shelf / whole store
Token Manager	Ops lead	CEO (as Token Admin) authorization	Ban addresses, emergency circuit-break TTS
Stake Admin / Manager	Ops lead	Community → CEO → further authorization	Mining rules (reserved)
Token Owner (brand)	The user who opened the shelf	Arises naturally	Adjust fee rate / power / chips

Key points:

- Contract roles are **"work keys," not personal property**;
- **The DAO Admin key is in the board's hands**, and the **CEO holds the Market Admin + Token Admin keys**;
- Each key's assignment is decided by a Category-I community proposal;
- Each key can be revoked by the community at any time (e.g. `setMarketAdmin(old_ceo, false)` , `setDAOAdmin(old_member, false)`);
- **Token Owner (brand)** is the exception — it's bound to "who opened this shelf," arises naturally, and isn't community-authorized.

11.3 The Board — the "Oversight + Arbitration" Pole in the Triangular Relationship

So far, we've covered two kinds of roles:

- **Community (DAO)**: TTS holders decide direction via proposals and votes;

- **Executive (CEO team)**: the CEO assembles three business teams responsible for landing.

If there were only these two layers, one problem remains: **is the CEO team really executing per the community's resolutions? Is the budget being used reasonably? Are the values being upheld?**

To answer this, beyond the "community – executive" binary structure, TTSWAP sets up a **third pole** — the **Board** — formed by **Partner Type-A TTS token holders.** Unlike a traditional public company's board that "decides everything and hires the CEO," the TTSWAP board **doesn't decide directly and doesn't execute business** — it only **oversees operations, arbitrates disputes, and guards TTSWAP's core values.**

Together with the other two parties it forms the **"community – executive – board" triangular relationship**:

Direction	Relationship
Community → Executive	Elect the CEO / grant contract keys / issue resolutions via passed proposals
Community → Board	Grant board seats / adjust board functions via proposals
Executive → Community	Report operations / execute per proposals / accept recall
Executive ↔ Board	Accept oversight / disclose data on request / respond to arbitration opinions
Board → Community	Submit reconsideration motions / submit oversight reports / call for emergencies

11.3.1 Why a "Third Pole" Is Needed

- **Oversee execution**: the CEO team holds the contract keys and drives daily work, but there needs to be an independent party continuously checking "whether they're doing things as agreed";
- **Arbitrate disputes**: when disputes arise in the community (proposal-interpretation disagreements, brand disputes, data-disclosure disputes), a body transcending interests needs to issue an opinion;
- **Guard values**: TTSWAP's core values (zero impermanent loss, one-token-one-pool, users can withdraw their funds) are the protocol's **bottom line** and mustn't be shaken by short-term interests;

- **Prevent "tyranny of the majority"**: pure holdings-weighted voting is easily dominated by whales — a **values filter** is needed.

> **Governance analogy**: the community is the legislature, the CEO team is the executive branch, and the board is like an "audit court + constitutional court" combination — **it doesn't execute specific business, but flags a yellow card on behavior that violates the constitution.**
>
> Important distinction: TTSWAP's "board" **isn't the "board that manages everything and hires the CEO" from Western corporate law**, but rather an **"independent supervision + values guardian"** role, closer to a combination of an audit committee / supervisory board + constitutional court.

11.3.2 Board Qualification and How Members Are Produced

- **Identity**: **Partner Type-A TTS token holders**;
- **Refined qualification**: the specific qualification criteria (e.g. Partner-share threshold, lockup requirements, behavior evaluation) are set by the community in later governance proposals. This whitepaper uses "Partner Type-A holder" as the generic name for this role without hard-coding specific thresholds;
- **How produced**: elected from qualified Partner-A holders; the specific number of directors, term, and election method are defined by a Category-I community proposal;
- **Independence**:
  - **The CEO cannot appoint or remove** any director;
  - A director's appointment / removal / adjustment **must pass a Category-I community proposal**;
  - A director must **proactively recuse** themselves on matters where they have a conflict of interest.

11.3.3 The Board's Five Core Responsibilities

Responsibility 1: Continuous Operational Oversight

- Oversee the reasonableness of the CEO team's **budget use** (public-sale funds, community treasury, marketing / R&D / operations budgets);
- Oversee whether **proposal execution** conforms to the original proposal text (executed on time, in the right amount, within scope);
- Review whether the code changes in a **contract upgrade** match the community proposal's description;
- Reconcile the ledger before and after operations involving the community treasury like `collectCommission(address(0))`.

Responsibility 2: Dispute Arbitration

- Issue arbitration opinions on **contract disputes** (e.g. whether a given automatic execution conforms to the proposal's semantics);
- Issue arbitration opinions on **brand disputes** (e.g. whether a shelf has a malicious config change);
- Issue arbitration opinions on **community disputes** (e.g. budget allocation, governance-parameter interpretation disagreements);
- An arbitration opinion isn't a final verdict — it's **the basis for triggering community re-deliberation.**

Responsibility 3: Values Guardianship and Reconsideration Power

- Initiate a **reconsideration motion** on **any** proposal that may violate TTSWAP's core values;
- "Values" include but aren't limited to:
  - The **zero-impermanent-loss LP return mechanism** mustn't be weakened;
  - The **one-token-one-pool + Singleton architecture** mustn't be forked and fragmented;
  - The **bottom line that users can withdraw their funds** (suppliers can withdraw anytime, customers can claim accumulated commissions anytime) mustn't be blocked;
  - The **six-party split summing to 100%** mustn't be breached;
  - The **one-time irreversibility of `disableUpgrade`** mustn't be bypassed in any way;
- Once the board initiates reconsideration, **the original proposal enters a cooling-off period** (e.g. an extra 72h), and the community must **re-vote**:
  - If the re-vote passes by **supermajority** → the original proposal takes effect;
  - Otherwise → the original proposal is voided.
- The board **cannot unilaterally veto** a proposal — the final decision still rests with the community: "**yellow card + reconsideration**" is not "red card + veto."

Responsibility 4: Transparency Requirements and Periodic Disclosure

- The board has the right to require the CEO team to disclose on an agreed cadence:
  - **Monthly report**: key operational metrics (TVL, trading volume, fee income, active address count, etc.);
  - **Quarterly report**: financial disclosure (public-sale fund balance, community treasury ledger, each team's budget use);
  - **Annual report**: a comprehensive governance report (proposal passage rate, emergency-response records, values-review conclusions).
- Disclosed content is **notarized on-chain** (IPFS / Arweave + on-chain hash reference), traceable long-term;

- If the CEO team fails to disclose on time, the board can initiate a Category-I proposal to hold them accountable.

Responsibility 5: Emergency Convening Power

- Upon discovering a **major anomaly** (suspected exploit, the CEO team collectively going dark, a major mis-operation), the board has the right to:
  - Request the CEO team to **respond urgently**;
  - Propose the community hold an **accelerated vote** (shortening the time to reach quorum);
  - In special cases, move to "**temporarily freeze** some of the CEO team's powers" (subject to a fast community vote for approval).
- The board **holds no contract keys itself** and can't call `freezeMarket` directly, but it can, via a proposal, have the community authorize a temporary executor to execute.

11.3.4 What It Doesn't Do (Clear Boundaries)

The board's **power boundaries** are equally clear:

What it doesn't do	Explanation
❌ Doesn't participate in specific business execution	Doesn't form teams or handle specific R&D / marketing / operations matters
❌ Doesn't hold Market Admin / Token Admin / Manager keys	Avoids mixing power and responsibility (holds only the DAO Admin key)
❌ Can't decide alone (except pure arbitration opinions)	All operations affecting protocol state must go through a community vote
❌ Can't recall the CEO alone	Recalling the CEO must go through a Category-I proposal
❌ Can't allocate budget alone	Budget use must go through a Category-III proposal

Core: the board's power is **"sunlight + a yellow card + a constitutional key,"** not "a scalpel." It influences decisions with public disclosure, arbitration opinions, and reconsideration motions; only after a community resolution authorizes it will it use the DAO Admin key to execute constitutional-level operations like "permanent lock / emergency mint-burn."

11.3.5 Examples of Interaction with the Community / CEO Team

Interaction Scenario A: Monthly Routine Oversight

```
Start of month:
  The CEO team publishes last month's "Operations Monthly Report" (TVL / volume / treas
  → The board reviews it
  → Spot-checks the on-chain evidence for 1–2 major operations
  → Publishes an "Oversight Brief," notarized on-chain
```

Interaction Scenario B: Budget Dispute

```
Trigger:
  A Category-III proposal passes, with budget for "hiring an external consultant for 50
  → After the CEO team executes, some community members question the selection process
  → The board steps in to investigate
  → The investigation conclusion is notarized on-chain
    · If compliant → publishes a "Compliance Confirmation"
    · If non-compliant → publishes an "Arbitration Opinion" → triggers community delibe
      (recover / hold accountable / revise the process)
```

Interaction Scenario C: Values Reconsideration

```
Trigger:
  The community passes a Category-II proposal to modify the "six-party split ceiling,"
  and the board believes the new config may over-dilute the LP share
  → The board initiates a "Values Reconsideration Motion"
  → The original proposal enters a 72h cooling-off period
  → The community re-votes
    · Passes by supermajority → the original proposal takes effect
    · Otherwise → the original proposal is voided
```

Interaction Scenario D: Emergency Convening

```
Trigger:
  The board receives an anonymous tip: the Market contract has a serious vulnerability
  → The board confirms credibility via internal deliberation within 1 hour
  → Publicly calls on the CEO team to respond immediately
  → Simultaneously initiates an "Emergency Vote Proposal" to the community: authorize t
  → After the community fast-vote passes, the executor executes lockGood / freezeMarket
```

11.3.6 Incentives and Accountability

- **Incentive**: directors receive a **fixed stipend from the community treasury** for their continuous work (periodic disclosure reviews, arbitration-opinion writing, reconsideration-motion handling), approved by a community budget proposal. The stipend is paid in TTS or USDT, notarized on-chain;
- **Accountability**:
  - If a director **fails their duty** (e.g. misses reviewing a major anomaly), the community can revoke that director's qualification via a Category-I proposal;
  - If a director **abuses the reconsideration power** (e.g. frequently initiating baseless reconsideration to delay governance), the community can restrict their reconsideration power or revoke qualification via a Category-I proposal;
  - In cases of **serious dereliction / self-dealing**, beyond revoking qualification, the community has the right to initiate punitive measures via a Category-IV proposal (e.g. burning their locked TTS share).
- **Exit**: a director can voluntarily exit at any time; after exiting they no longer receive the stipend, but the stipend for work already completed is still paid.

11.3.7 Contract-Layer Expression

The board's direct correspondence in the contract is the **`DAO Admin` key** (`UserConfig[255]`) — i.e. the highest permission bit. The board holds the DAO Admin key via multisig, used to execute the following on-chain actions (all authorized by community resolution):

- `disableUpgrade()` : **permanently lock contract upgrades** (one-time, irreversible — the trustless endgame);
- `setDAOAdmin / setTokenAdmin / setMarketAdmin / setStakeAdmin` : appoint or revoke the next-level Admin per the result of a Category-I community proposal;
- `mint(to, amount) / burn(from, amount)` : **emergency mint/burn TTS** (contract-migration compensation / vulnerability fix), executed only after a Category-IV community proposal passes;
- `fixLeftShare()` : accounting correction;
- `setEnv()` : designate the Market contract address at deployment.

Beyond the above "constitutional-level" on-chain keys, **the board's daily oversight / arbitration / reconsideration functions currently operate mainly via "off-chain institution + on-chain event notarization,"** unlike the pure execution bitmaps of Admin / Manager.

Implementation (phased, gradually on-chain):

1. **Phase 1 (off-chain-led)**: the board publishes arbitration opinions / disclosure briefs via off-chain tools (Snapshot / multisig forum), with key conclusions notarized as on-chain hashes; the `DAO Admin` key is used only when executing hard resolutions like "permanent lock / emergency mint-burn";
2. **Phase 2 (semi-on-chain)**: the governance contract introduces a "reconsideration lock" field — the board multisig can set a reconsideration lock on a given proposalId, triggering a 72h cooling-off period; the community must re-vote to unlock;
3. **Phase 3 (fully on-chain)**: the board contract goes live as an independent module of the governance system, interconnected with the main governance contract, with the `DAO Admin` key and "reconsideration lock" realizing a **fully automatic "oversight + arbitration + reconsideration"** flow.

> **Current version**: the whitepaper describes the target institution; the board's specific contractization details will be confirmed by the community as **subsequent governance proposals.**

11.4 Typical Governance Cases

Case A: New Good `$GEM` Elevated to Hard Currency

```
Background:
  $GEM has been listed for 3 months, TVL 1M USDT, daily volume ~300K USDT,
  and community discussion concludes it can be elevated to hard currency.

Flow:
  1. The $GEM brand + marketing team draft the proposal ("elevate $GEM to hard currency
  2. The proposer submits a Category-II proposal, meeting the TTS holding threshold;
  3. The DAO enters a 72h voting period;
  4. The vote passes (majority for + quorum met);
  5. The Timelock cooling-off period (assume 48h) ends;
  6. The governance-contract executor automatically calls modifyGoodByAdmin(GEM, 2**255
  7. The e_modifyGoodConfig event is broadcast, and $GEM enjoys hard-currency privilege

The whole process is on-chain auditable; the CEO doesn't directly approve, only coordin
```

Case B: Emergency Vulnerability Loss-Stop (off-chain coordination + after-the-fact ratification)

Background:
A white-hat researcher reports via Immunefi that a branch of the Market contract has that could be exploited to drain a specific shelf's liquidity.

Emergency flow:
1. White-hat reports → the operations team receives the alert immediately;
2. The operations team reproduces it within 15 minutes and confirms the danger;
3. The CEO convenes an emergency R&D + operations meeting;
4. The CEO authorizes the R&D team to call freezeMarket(), pausing trading across the
5. The R&D team develops the V2 implementation + submits it for audit;
6. After the audit passes, the CEO initiates a Category-IV proposal "upgrade to V2 an
7. The DAO passes it (possibly via the accelerated channel, shortening the voting per
8. The governance-contract executor calls upgrade(V2) → service restored.
9. The CEO submits a complete incident report to the DAO, and the DAO ratifies the em

Design essence: the emergency brake is at the "executive layer," but the long-term fix

Case C: The Trustless Endgame

Background:
The contract has run for 2 years + 3 external audits + 0 major incidents, and the com whether to trigger disableUpgrade to complete the final trustless step.

Flow:
1. Months of community discussion + solicitation of external audit-expert opinions;
2. The CEO and R&D team complete a final round of code freeze;
3. The proposer submits a Category-IV proposal "permanently lock the contract-upgrade
4. The DAO enters a 14-day voting period + 7-day Timelock;
5. Passes by supermajority (≥ 2/3);
6. The governance-contract executor calls disableUpgrade();
7. From then on, upgrade / freezeMarket / freezeToken are all abolished;
8. The contract logic is permanently immutable.

Afterward the CEO's responsibilities remain (external collaboration / brand operations) "keys" at the contract layer all become void, and the protocol enters a truly "bossless

11.5 The Checks-and-Balances Principles Behind the Architecture

A decentralized mall isn't "unmanaged" — it's "**no one can call the shots alone.**" TTSWAP's separation-of-powers architecture guarantees this with four checks:

Check 1: Separation of Decision Power and Execution Power

- **Decision power** belongs to the DAO (TTS holders);
- **Execution power** belongs to the CEO team (via holding the contract Admin / Manager keys);
- The CEO can't unilaterally initiate resolutions on organization/personnel / protocol changes / budget / major matters — **these must go through proposals**;
- The DAO can recall the CEO and revoke keys at any time via a new proposal.

Check 2: The Board (DAO Admin) Is Independent of the Executive (Market/Token Admin)

- The **board (holding the DAO Admin key)** doesn't hold business keys like Market Admin / Token Admin / Manager;
- The CEO (holding Market Admin + Token Admin) has no power to appoint or remove board members;
- On proposals that violate core values, the board has a "reconsideration power" (yellow card + cooling-off + community re-vote), but **can't unilaterally veto**;
- This guarantees that even if a community vote is hijacked by short-term interests, there's still a "values filter" layer.

Check 3: Separation of Emergency Response and Permanent Resolution

- The **emergency buttons** (`freezeMarket` / `freezeToken` / `lockGood`) are in the CEO team's hands, responsive at millisecond speed;
- The **permanent buttons** (`disableUpgrade` / upgrading the contract to a brand-new version) must go through a DAO resolution + Timelock;
- Even if a CEO-team member impulsively presses an emergency button, at most it **pauses service** — it won't cause **irreversible consequences.**

Check 4: External Participants Are Ungoverned

- Suppliers, customers, merchants, referrals, storefronts, and brands **need no governance authorization whatsoever**;

- Even if the CEO team collectively neglects their duty, the community proposal system is paralyzed, and the board has problems too, ordinary users can still:
  - Withdraw their own supply (`disinvestProof`);
  - Buy and sell goods (`buyGood` / `payGood`);
  - Claim accumulated commissions (`collectCommission`).
- **This layer is hard-coded in the contract; no governance resolution has the power to block the withdrawal of user funds.**

11.6 Architecture Evolution Path (from Founding to Endgame)

Most decentralized protocols go through the following three phases, and TTSWAP is no exception:

Phase	Decision architecture	Organizational structure	Board	Landmark actions
Phase 1: Founding	Founding-team multisig + early whitelisted holders participate	Founding team doubles as CEO + three teams	Off-chain-led, key conclusions notarized on-chain via hash; the DAO Admin key is in the founding team's multisig	Public sale, first hard-currency elevation, first goods listed
Phase 2: Stable Operations	Community proposals become the norm; the CEO is produced by a formal vote	The CEO-assembled three-team system is institutionalized	The governance contract introduces the "reconsideration lock" field (semi-on-chain); the first Partner-A board election completes and takes over the DAO Admin multisig	Establish Timelock, formal governance contract, board institutionalization

Phase	Decision architecture	Organizational structure	Board	Landmark actions
Phase 3: Trustless Endgame	`disableUpgrade` triggered; the contract is permanently immutable	The CEO becomes a "nominal" role; the protocol self-runs	The board contract goes live as an independent module; after `disableUpgrade` executes, the DAO Admin key becomes permanently void	All contract Admin / Manager keys become void

Evolution cadence:

- Phase 1 → Phase 2: usually 6–12 months, marked by **a complete, voted-in CEO appointment** and **the first Partner-A board election completing and taking over the DAO Admin multisig**;
- Phase 2 → Phase 3: usually 2–3 years, marked by **at least 3 external audits + 0 major incidents + a community supermajority resolution**;
- After Phase 3: the protocol enters the ultimate state of "**code is the constitution**," where users depend only on the contract itself, no longer on any team.

12 The TTS Token: The Mall's Scarcest "Contribution Certificate"

12.1 TTS Isn't Vapor — It's the Mall's Virtual Equity and More

Many DeFi protocols' governance tokens are essentially just "voting certificates," decoupled from actual protocol use. TTSWAP's TTS token is different:

> **TTS token = this mall's virtual equity + voting rights + market-cap-driven unlock certificate.**

A few of TTS's core attributes:

- **Name / symbol**: TTSwap Token / **TTS**;

- **Precision**: 12 decimals;
- **Max supply**: **200 million**;
- **Inflation mechanism**: ~50-year linear release, all distributed to suppliers (§12.2);
- **Deflation mechanism**: 100% of the mall's "community treasury" fee portion buys back and burns TTS (§12.4).

12.2 Supply-as-Mining: TTS Arrives Automatically

Each time someone calls `investGood` to restock a **commitment-type** shelf, the protocol converts that batch's "value" into TTS hash power and records it automatically.

Every 86,400 seconds (1 day) the protocol auto-settles emission once:

```
Daily emission = (200M − current circulating supply) / 18250 × chainRatio / 10000
(when the result is below 1 TTS, a floor of 1 TTS applies)
```

Intuition:

- The less remaining issuable supply there is, the smaller the daily emission;
- The emission is distributed to different chains by "the current chain's trading share" `chainRatio` ;
- Linear release completes in ~50 years;
- **As long as a supplier has goods on a shelf, TTS keeps arriving automatically** — no "claim," no extra operation needed.

12.3 4C Allocation: How Founding Shareholders Split the Initial TTS

The initial 45 million TTS (`left_share`) is reserved for four kinds of contributors, called the **4C model**:

Role	Share	Notes
Founder	20%	Project initiators
Partner	12%	Core team
Value Contribution (Value)	44%	Community operations, technical development
Capital Contribution (Capital)	24%	Early investors, airdrops, public sale

4C design philosophy: give "value contribution (people who do the real work)" the largest share, while capital that comes in actually takes a relatively small share. This is deliberate — **labor over capital.**

12.4 Doubling Unlock: No Market-Cap Doubling, No Token Unlock

Every 4C role's TTS is **fully locked**, and unlocking must satisfy a strict condition:

> **Only when the mall's TTS price relative to USDT reaches the set threshold can a portion be unlocked. After each unlock, the threshold doubles.**

The on-chain judgment logic (simplified):

```
function shareMint() external {
    // The current TTS/USDT price must be higher than 2^(times unlocked) × the starting
    if (!market.ishigher(TTS, USDT, 2^metric × 2^128 + 20_000_000))
        revert;
    // Unlock amount = remaining locked amount / chips
    uint128 mintamount = leftamount / chips;
    shares[msg.sender].leftamount -= mintamount;
    shares[msg.sender].metric += 1;
    _mint(msg.sender, mintamount);
}
```

Translated into mall-speak:

> "I'm a founding shareholder holding 1 million TTS, but all locked.
> Only when the TTS price relative to USDT rises above the threshold for the Nth unlock can I unlock a portion.
> Each unlock doubles the threshold. So **if the price never rises, my TTS stays locked forever.**"

The per-unlock cap is determined by each 4C role's `chips` parameter:

Role	Share	chips	Per-unlock cap	Trigger example
Founder	20%	20	1/20 = 5%	1000× rise → cumulative unlock ≈ 37%

Role	Share	chips	Per-unlock cap	Trigger example
Partner	12%	14	1/14 ≈ 7.1%	400× rise → cumulative unlock ≈ 45%
Value Contributor	44%	12	1/12 ≈ 8.3%	200× rise → cumulative unlock ≈ 50%
Capital Contributor	24%	8	1/8 = 12.5%	100× rise → cumulative unlock ≈ 60%

Intuition: **Founders and Partners are the project's value maintainers, bound to the mall long-term, so their `chips` are the largest and their per-unlock ratio the smallest (Founder 5%, Partner ≈ 7.1%) — only as the mall delivers sustained long-term value round after round do their tokens trickle out tier by tier**, ensuring those who understand the protocol best don't jump ship early during a short-term surge. Value Contributors unlock one tier faster (smaller `chips`, ≈ 8.3% per unlock), encouraging "people who do things" to cash out sooner. Capital Contributors have the lowest unlock threshold and the highest per-unlock cap (1/8 = 12.5%, 100× rise → cumulative ≈ 60%), but the smallest total share — capital puts up money and recoups it fast, but doesn't occupy a big slice of the pie.

12.5 The Three Economic Implications of 4C Doubling Unlock

Implication 1: If the price doesn't rise, tokens never unlock

This is the ultimate "**anti-dump**" design: traditional VC tokens often "unlock upon listing," bringing huge sell pressure. TTSWAP binds unlocking to price appreciation — **only when the TTS price rises do tokens get a chance to be cashed out.**

Implication 2: Token release cadence = market-cap growth cadence

The unlock process is like a **feedback loop**:

- The better the mall's business → the more TTS bought back and burned → the higher the TTS price;
- The higher the TTS price → the more tokens unlock → the founding team / investors can gradually cash out.

This feedback loop gives **token release its own "market-validated" cadence**, without needing a DAO vote to "slowly release the tap."

Implication 3: No need for continuous governance voting

Many token economics need governance votes to unlock and to adjust supply, easily falling into a "tragedy of the commons" or whale manipulation. TTSWAP's doubling unlock is **mechanized governance — the code executes it itself**, with no need for N governance votes a year.

12.6 Public Sale: Tiered Pricing + Hard Cap

Beyond the 4C allocation, TTSWAP reserves a portion of TTS for a **public raise**:

Tier	USDT raised	Exchange rate
Tier 1	≤ 87,500	1 USDT = **25 TTS**
Tier 2	87,500 – 162,500	1 USDT = **20 TTS**
Tier 3	162,500 – 250,000	1 USDT = **16 TTS**

- **Hard cap 250,000 USDT** — exceeding it reverts directly;
- On-chain transparent — anyone can check the public-sale progress in real time;
- The USDT raised is withdrawn by the Token Admin via `withdrawPublicSell` for protocol startup funds.

12.7 Offline-Signed Batch Allocation (`permitShare`)

If a **large-scale airdrop** / **batch settlement with partners** is needed, the TTS operations director can use EIP-712 to offline-sign a batch of allocation orders, and the signees claim them on-chain themselves.

Benefits:

- **Batch, asynchronous, zero-Gas** signing;
- Suited for airdrops, partner quotas, community incentives;
- All allocations leave event logs on-chain — fully auditable.

12.8 Inflation vs. Deflation: A Dual Engine

TTSWAP's TTS economy is a **dual-engine motor**:

Engine	Direction	Mechanism
Inflation	Issuance	Daily ~ `(200M − totalSupply) / 18250 × chainRatio / 10000` emitted to LPs
Deflation	Burn	100% of the mall's community-treasury profit buys back TTS on the secondary market and burns it

Long-term dynamics:

- Early: few suppliers, inflation > deflation → incentivize mining;
- Growth: volume amplifies, inflation < deflation → circulating supply converges;
- Maturity: many new brands onboard, TTS scarcity keeps rising.

This design keeps TTS from falling into the "the more issued, the more it depreciates" trap that plagues most DeFi tokens.

13 Upgrades, Circuit Breakers, and Permanent Immutability

TTSWAP's contracts use the **proxy pattern** (`TTSwap_Market_Proxy` / `TTSwap_Token_Proxy`):

- **Front desk** (Proxy): stores all state (supply certificates / shelf inventory / user balances);
- **Back kitchen** (Implementation): stores only business logic;
- The front desk forwards requests to the back kitchen for execution via `delegatecall` .

Benefit: **the back kitchen can be swapped while the front desk stays put, and all state carries over.**

Three key operations:

Operation	Permission	Notes
`upgrade(newImpl)`	**CEO** (holds Market Admin / Token Admin key)	Upgrade the mall's capabilities
`freezeMarket` / `freezeToken`	Admin / TTS operator	In emergencies, set the back kitchen to 0 so all operations revert (**emergency**

Operation	Permission	Notes
		brake)
`disableUpgrade`	**Board** (holds the DAO Admin key)	**Permanent lock** — afterward `upgrade` and `freeze` are all abolished

Separation of two powers: the daily upgrade power (Market/Token Admin) is in the CEO's hands (fast response), while the permanent-lock power (DAO Admin) is in the board's hands (the trustlessness commitment).

14 Common Scenario Cheat Sheet

Scenario	Which function	Which section
List a new good (founder)	`initGood`	§4
Restock an existing shelf (supplier)	`investGood`	§5
Ordinary user buys something (customer)	`buyGood`	§6.1
Exact payment (buyer or merchant)	`payGood`	§6.2
Merchant collects in USDT (zero slippage)	`payGood` (same-token fast path)	§6.3
Sponsor Gas / order for a token-less user	X402 signature mode	§7
Supplier withdraws / takes profit	`disinvestProof`	§9
Supplier / storefront claims accumulated commissions	`collectCommission`	§10.5
Brand adjusts fee rate / amplification	`modifyGoodByGoodOwner`	§11.2
Batch several actions	`multicall`	§6
Elevate a good to hard currency	`modifyGoodByAdmin` (CEO / Market Admin)	§11.2

Scenario	Which function	Which section
Emergency-freeze a shelf	`lockGood` (brand or admin)	§11.3
Unlock a portion of TTS	`shareMint` (needs price trigger)	§12.4
Participate in the public sale	`publicSell`	§12.6
Sign a batch TTS airdrop	`permitShare`	§12.7

15 Contact & Legal License

15.1 Contact

- **Website**: https://ttswap.io
- **Twitter**: @ttswapfinance
- **Telegram**: @ttswapfinance
- **Discord**: ttswap
- **Email**: bussiness@ttswap.io
- **GitHub**: github.com/ttswap

15.2 Legal License

File category	License	Meaning
Core contracts (Market / Token / Proxy / L_Good / L_Proof)	**BUSL-1.1**	Business Source License, forbidding commercial forks for a specified term
Interfaces and utility libraries (interfaces / L_Error / L_GoodConfig / L_TTSwapUINT256 …)	**MIT**	Permissive open source, easy for community integration
ERC20 base implementation	**AGPL-3.0-only**	Copyleft open source
EIP-1153 transient-storage library	**GPL-3.0-or-later**	Copyleft open source

File category	License	Meaning
WETH interface	**GPL-2.0-or-later**	Copyleft open source

> **Commercial-use notice**: if you intend to fork TTSWAP for a commercial product, please first read the `LICENSE` file in the GitHub repository and contact `bussiness@ttswap.io` for authorization.

16 Appendices

Appendix A: Mall Story ↔ Protocol Concept Mapping

Role / object in the mall story	Protocol correspondence	Code / contract location
The whole mall	The TTSWAP protocol	—
Central checkout	The Singleton contract	`TTSwap_Market_Proxy`
Shelf	Each good's pool	`mapping(goodid => S_GoodState)`
Good identifier	`T_GoodKey` (type + address + sub-id)	`toId()` → goodid
Hard-currency good	Value Good	Elevated via `modifyGoodByAdmin`
Normal good	Normal Good	Default
Current inventory Q	`currentState.amount1`	—
Cumulative stock-in I	`currentState.amount0`	—
Market value V	`investState.amount1`	—
Total supply-certificate shares S	`investState.amount0`	—
Supply certificate	Proof of Investment	`proofs[proofid]`
Virtual display depth	enpower	`getInvestPower()`

Role / object in the mall story	Protocol correspondence	Code / contract location
Swap-curve coefficient	Fixed at 2	`buyGoodInput/Output`
Shelf-depth coefficient M	`power`	`goodConfig` user zone
Safe line	safeLine	`TTSwapError(55/56)`
Brand	Good Owner	`goods[goodid].owner`
Admin	Market Manager	`UserConfig[251]`
CEO	**Market Admin**	`UserConfig[252]`
TTS operator / director	Token Manager / Token Admin	`UserConfig[253/254]`
Board	**DAO Admin**	`UserConfig[255]`
Merchant	Executor	X402 / EIP-712 signature relayer
Execution fee	`executeFee`	Contract constant `5e10` (≈ 0.05 USDT)
Supplier / restock	LP / `investGood`	—
Customer / place order	Trader / `buyGood` or `payGood`	—
Same-token direct payment	`payGood` same-token fast path	`_goodid1 == _goodid2`
Merchant subsidy	`goodWelfare`	—
The mall's TTS token	TTS Token	`TTSwap_Token.sol`

Appendix B: Key Parameter Limits (Contract Hard Constraints)

Parameter	Upper / lower bound	Meaning
Initial quantity	$[500{,}000, 2^{109}]$	Inventory range for opening a new shelf

Parameter	Upper / lower bound	Meaning
Initial custom value	$[5 \times 10^{14}, 2^{109}]$	Total-value range for opening a new shelf
Minimum additional-investment value	$\geq 10^{12}$	Minimum value for a single `investGood`
Minimum trade value	$\geq 10^{8}$	Minimum transferred value for a single trade
Quantity overflow protection	$< 2^{109}$	Overflow ceiling for any shelf's quantity field
Execution fee `executeFee`	5×10^{10} (about 0.05 USDT)	Fixed constant
Public-sale hard cap	2.5×10^{11} (250,000 USDT)	One-time, unbreakable
Public-sale tiers	25 / 20 / 16 TTS	How many TTS per USDT
TTS total supply	2×10^{20} wei = 200 million TTS	Permanent ceiling
Daily-emission calc period	86,400 seconds	`_stakeFee` triggers once daily
Commission batch limit	≤ 100	`collectCommission` claims at most 100 shelves at once

Appendix C: A One-Chart Comparison with Mainstream DeFi

Dimension	Uniswap V2	Uniswap V4	Curve	Aave	TTSWAP
Architecture	One contract per pair	Singleton + Hook	Single pool	Lending pool	**Singleton one-token-one-pool**
Is LP two-token?	Required	Required	Single or dual	Single	**Single**

Dimension	Uniswap V2	Uniswap V4	Curve	Aave	TTSWAP
Impermanent-loss protection	None	None	Weak	N/A	**Protocol-level + triple defense**
Liquidity aggregation	Fragmented by pair	Partial	Multi-pool	N/A	**Whole-protocol aggregation**
Six-party commission	LP only	LP + protocol + Hook	LP + veCRV	Depositor + protocol	**LP + brand + storefront + referral + customer + mall**
Native ETH	No	Yes	No	No	**Yes**
Same-token exact payment	No	No	No	No	**Yes (fast path)**
Sponsored Gas (token-less user)	No	Partial	No	No	**Native X402**
Token economics	Governance	Governance	veCRV (lockup)	Governance + safety module	**Supply-as-mining + 4C doubling unlock + 100% buyback-and-burn**

Appendix D: Glossary

- **Shelf (pool)**: each good's unique pool in TTSWAP, corresponding to `S_GoodState`;
- **Supply certificate (Proof)**: the on-chain ledger entry produced by each restock;
- **Hard-currency good (Value Good)**: a good elevated by the community to be a "pricing anchor";
- **Normal good (Normal Good)**: all other new goods, constrained by safeLine;
- **CEO (Market Admin + Token Admin)**: the top executive officer produced by a community vote, holding the good-config and contract-upgrade keys;

- **Admin (Market Manager)**: the admin for daily operations / commissions / freezing shelves;
- **Board (DAO Admin)**: formed by Partner Type-A holders, holding the DAO Admin key, responsible for overseeing operations, arbitrating disputes, and guarding values, and controlling `disableUpgrade` / emergency mint-burn;
- **Merchant (Executor)**: the X402 signature relayer;
- **Execution fee (executeFee)**: `5e10` , about 0.05 USDT;
- **TTS**: TTSWAP's governance / points token, with a total supply of 200 million;
- **4C model**: the allocation model for the four kinds of contributors — Founder / Partner / Value / Capital;
- **Doubling unlock**: for each doubling of the TTS price relative to USDT, a portion of the locked tokens unlocks;
- **Six-party split**: the automatic fee distribution among the six parties — LP / Operator / Gate / Referral / Customer / Platform.

End of Whitepaper · TTSWAP Decentralized Mall · 2026-07-07

> For any questions or partnership needs, please contact bussiness@ttswap.io.
> This document is a protocol description only and does not constitute any investment advice; please fully assess the risks before participating in a DeFi protocol.